

Московский физико-технический институт
(национальный исследовательский университет)

Физтех-школа прикладной математики и информатики
Кафедра математических методов в физике

Нейронные сети в прайсинге производных финансовых инструментов

*Ускорение калибровки модели грубой волатильности
с помощью операторных нейронных сетей на GPU*

Направление: 01.04.02 Прикладная математика и информатика
Студент: Бирюков Никита Андреевич
Научный руководитель:
Рецензент:

Москва, 2026

Abstract

Тема: Нейронные сети в прайсинге производных финансовых инструментов.

Цель работы: Разработка и исследование системы быстрой калибровки модели грубой волатильности Heston (Rough Heston) с использованием операторных нейронных сетей (Fourier Neural Operator, FNO) и автоградиентного метода Ньютона–Гаусса.

Методы: GPU-реализация метода Фурье–COS для точного прайсинга поверхности подразумеваемой волатильности; суррогатная модель FNO с FiLM-кондиционированием (Feature-wise Linear Modulation) по параметрам модели; метод Ньютона–Гаусса с якобианом, вычисленным через прямое автоматическое дифференцирование (forward-mode AD, `jacfwd`).

Основные результаты:

1. Реализован GPU-ускоренный прайсер (exponential-midpoint RK4), точность ≤ 10 б.п. при $N = 40$ факторах Бернштейна.
2. Обучена суррогатная модель FNO v2 (50 000 образцов, $R^2 = 0,9991$, MAE = 0,058%), инференс < 1 мс.
3. Метод Ньютона достигает сходимости за 5–7 итераций ($p_{95} = 668$ мс), что в $3\times$ быстрее L-BFGS при ошибке $v_0 < 2\%$.
4. Продемонстрирована реал-тайм-калибровка с частотой 1,7 сделки/с ($p_{95} = 668$ мс < 1000 мс).
5. Дельта-хеджирование с FNO-калибровкой снижает дисперсию P&L на 5,1% по сравнению с плоской волатильностью Блэка–Шоулса.
6. Расширение пространства параметров: обучена модель FNO v3 с обучаемым показателем Хёрста $H \in [0,04; 0,15]$.

Практическая значимость: Вся система работает в реальном времени на потребительском GPU (NVIDIA RTX 3060), что делает её применимой для задач риск-менеджмента и маркет-мейкинга на рынках опционов.

Ключевые слова: грубая волатильность, Rough Heston, операторные нейронные сети, FNO, FiLM, прайсинг деривативов, калибровка, метод Ньютона, автоматическое дифференцирование, показатель Хёрста.

Contents

Введение	6
1 Обзор литературы: Смена парадигмы в количественных финансах	9
1.1 Ведущие исследовательские группы и их вклад	9
1.1.1 Группа Deep BSDE (Принстон, ETH Zurich)	9
1.1.2 Группа DGM (University of Oxford, UIUC)	10
1.1.3 Группа Deep Hedging (JP Morgan, ETH Zurich)	11
1.1.4 Группа Deep Calibration (King's College London, Imperial College)	11
1.1.5 Группа прикладных исследований (TU Delft, CWI, NYU)	12
2 Архитектуры нейронных сетей в финансовом моделировании	12
2.1 Полносвязные сети (Feed-Forward Networks / MLP)	13
2.1.1 Теоретическая справка	13
2.1.2 Применение в финансах	13
2.1.3 Преимущества и недостатки	13
2.2 Остаточные сети (Residual Networks, ResNets)	13
2.2.1 Теоретическая справка	13
2.2.2 Применение в финансах	14
2.2.3 Преимущества и недостатки	14
2.3 Рекуррентные сети (RNN / LSTM)	14
2.3.1 Теоретическая справка	14
2.3.2 Применение в финансах	14
2.3.3 Преимущества и недостатки	14
2.4 Сравнительный анализ архитектур	15
3 Детальный анализ ключевых исследований	15
3.1 Метод Деер BSDE: Стохастическая оптимизация для высокоразмерных PDE	15
3.1.1 Проблема и мотивация	15
3.1.2 Математическая теория	16
3.1.3 Алгоритм и применение нейросетей	16
3.1.4 Область применения и преимущества	17
3.2 Метод DGM: Глобальное решение и задачи со свободной границей	17
3.2.1 Проблема и мотивация	17
3.2.2 Математическая теория	18
3.2.3 Алгоритм и Архитектура	18
3.2.4 Область применения и преимущества	18
4 Детальный анализ ключевых исследований	19
4.1 Метод Деер BSDE: Стохастическая оптимизация для высокоразмерных PDE	19
4.1.1 Проблема и мотивация	19

4.1.2	Математическая теория	20
4.1.3	Алгоритм и применимость	20
4.2	DGM: Бессеточный метод Галёркина	20
4.2.1	Проблема и мотивация	20
4.2.2	Математическая теория	21
4.2.3	Алгоритм и применимость	21
4.3	Deep Hedging: Оптимизация торговых стратегий при наличии трений . .	21
4.3.1	Проблема и мотивация	21
4.3.2	Математическая теория	21
4.3.3	Алгоритм и применимость	22
4.4	Deep Calibration: Ускорение калибровки сложных моделей	22
4.4.1	Проблема и мотивация	22
4.4.2	Суть метода	22
4.4.3	Алгоритм и применимость	22
4.5	CaNN: Калибровка через механизм обратного распространения ошибки .	22
4.5.1	Суть метода	23
4.5.2	Применимость	23
4.6	Теоретическое обоснование: Рандомизированные сигнатуры	23
4.6.1	Суть метода	23
4.6.2	Применимость	23
4.7	Инженерные аспекты: Обеспечение финансовой корректности	23
4.7.1	Проблема и решения	23
4.7.2	Применимость	24
5	Сравнительный анализ и Выводы	24
5.1	План практического исследования	24
6	Математические основы	25
6.1	Рынок опционов и подразумеваемая волатильность	25
6.2	Дробное броуновское движение и грубая волатильность	25
6.3	Модель Rough Heston	26
6.4	Проблема идентифицируемости	26
6.5	Лифтинговое представление и дробное уравнение Риккати	27
6.5.1	Приближение Бернштейна (лифтинг)	27
6.6	Условия отсутствия арбитража для поверхностей IV	27
7	GPU-ускоренный прайсинг методом Фурье–COS	29
7.1	Метод Фурье–COS	29
7.2	Решение лифтинговой системы ОДУ	29
7.2.1	Exponential Midpoint Integrator	29
7.3	GPU-реализация: пакетная обработка	30
7.4	Сходимость по числу факторов Бернштейна	30

7.5	Сходимость по числу шагов ODE	31
7.6	Генерация датасета	32
8	Суррогатная модель: FiLM-кондиционированный FNO	33
8.1	Мотивация: зачем FNO, а не MLP?	33
8.2	Архитектура FNO	33
8.2.1	Зеркальное дополнение (Mirror Padding)	33
8.3	FiLM-кондиционирование	34
8.4	Полная архитектура MirrorPaddedFNO2d	34
8.5	Нормализация	34
8.6	Функция потерь	35
8.7	Обучение	35
9	Быстрая калибровка: FIM и метод Ньютона–Гаусса	36
9.1	Постановка задачи калибровки	36
9.2	Анализ идентифицируемости через матрицу Фишера	36
9.2.1	FIM-эллипсоид доверия	36
9.3	Репараметризация $6D \rightarrow 3D$	37
9.4	Метод Ньютона–Гаусса с <code>jacfwd</code>	37
9.4.1	Вычисление якобиана через forward-mode AD	37
9.5	Регионы начального приближения и мульти-старт	38
9.6	Расширение до 4D: обучаемый показатель Хёрста	38
10	Экспериментальные результаты	39
10.1	Точность суррогатной модели FNO v2	39
10.1.1	Датасет и протокол	39
10.1.2	Результаты	39
10.2	Сравнение методов оптимизации: Newton vs L-BFGS	40
10.2.1	Протокол	40
10.2.2	Результаты	40
10.3	Потоковая (streaming) калибровка в реальном времени	41
10.3.1	Постановка	41
10.3.2	Результаты	41
10.4	Хеджирование по дельте с учётом улыбки волатильности	42
10.4.1	Мотивация	42
10.4.2	Протокол бэктеста	42
10.4.3	Результаты	43
10.5	Расширение: обучаемый показатель Хёрста	43
10.5.1	Мотивация	43
10.5.2	FNO v3: архитектура и датасет	43
10.5.3	Результаты обучения и оценки FNO v3	44
10.5.4	4D-калибратор с обучаемым H	45

Заклучение	46
Список литературы	49

Введение

Мотивация

Современный рынок биржевых и внебиржевых опционов ежедневно генерирует колоссальный поток данных о ценах. Маркет-мейкер, управляющий книгой опционов на S&P 500, вынужден калибровать стохастическую модель волатильности по реальной поверхности подразумеваемой волатильности (implied volatility, IV) в режиме реального времени — за секунды после каждого изменения котировок. Аналогичная задача возникает при управлении риском (вычисление греков), XVA-вычислениях и стресс-тестировании портфелей.

Классические модели — Black–Scholes (1973) [1], Heston (1993) [2], SABR (2002) [3] — допускают аналитические или полуаналитические решения, однако недостаточно гибки для описания эмпирических феноменов:

- **Наклон улыбки волатильности** (volatility skew) при коротких сроках до погашения;
- **Степенное поведение АТМ-волатильности** вида $\sigma_{\text{ATM}}(T) \sim C \cdot T^{H-1/2}$ с показателем Хёрста $H \approx 0,1$, характерным для дробного броуновского движения (Gatheral et al., 2018) [4];
- **Долгую память** волатильности: автокорреляция реализованной волатильности убывает по степенному закону, а не экспоненциально.

Модели *грубой волатильности* (Rough Volatility), в частности **Rough Heston** (El Euch & Rosenbaum, 2019) [5], воспроизводят эти свойства, управляя дисперсионным процессом дробным броуновским движением с $H < 1/2$. Однако цена этой реалистичности — **вычислительная сложность**: калибровка Rough Heston классическими методами (Монте-Карло, численное решение дробных уравнений Риккати) занимает от нескольких минут до нескольких часов, что делает их неприменимыми для реального трейдинга.

Цель и задачи работы

Цель работы — разработать систему быстрой калибровки модели Rough Heston, которая обеспечивает **латентность менее 1 секунды** при **точности, сопоставимой с референсным решателем**, и исследовать её применимость для задач хеджирования и реал-тайм риск-менеджмента.

Для достижения цели решались следующие **задачи**:

1. Реализовать GPU-ускоренный **прайсер Фурье–COS** с устойчивым ODE-решателем (exponential midpoint) для генерации датасетов поверхностей IV.
2. Разработать и обучить **суррогатную модель** на основе Fourier Neural Operator (FNO) с кондиционированием по параметрам через Feature-wise Linear Modulation (FiLM).

3. Построить **быстрый калибратор** на основе метода Ньютона–Гаусса с автоматическим дифференцированием (forward-mode AD), используя суррогат как оракул функции и якобиана.
4. Провести **теоретический анализ идентифицируемости** параметров через матрицу Фишера (FIM) и обосновать 3D-репараметризацию.
5. Расширить пространство параметров, включив **обучаемый показатель Хёрста** H , и оценить влияние на качество калибровки.
6. Продемонстрировать **практическую применимость**: реал-тайм потоковая калибровка, авторегрессионное хеджирование по дельте.

Научная новизна

1. **FiLM-FNO для прайсинга**: применение FNO с FiLM-кондиционированием к задаче аппроксимации поверхности IV Rough Heston достигает $R^2 = 0,9991$, MAE = 0,058% при инференсе < 1 мс — в 10^3 - 10^4 раз быстрее прямого прайсинга.
2. **Автоградиентный метод Ньютона**: первое применение jacfwd (forward-mode AD через FNO) для вычисления точного якобиана $\partial IV / \partial \theta$ за 3 прохода вместо 10 при конечных разностях, с квадратичной сходимостью к минимуму.
3. **Идентифицируемость через FIM**: доказана численно разница числа обусловленности $\kappa(\mathbf{I}) = 1301 \times$ между 6D и 3D параметризацией, обосновывающая фиксацию κ, θ, H .
4. **Обучаемый показатель Хёрста**: расширение 3D-калибратора до 4D с включением $H \in [0,04; 0,15]$ как свободного параметра.

Практическая значимость

Все вычисления выполняются на потребительском GPU NVIDIA RTX 3060 (6 ГБ VRAM). Медианная латентность калибровки 549 мс, $p_{95} = 668$ мс — достаточно для обработки тиков SPX-опционов (≈ 1 – 2 секунды между тиками) в реальном времени.

Структура работы

- Глава 1.** *Математические основы*: модель Rough Heston, дробное броуновское движение, лифтинговое представление, условия отсутствия арбитража для поверхностей IV.
- Глава 2.** *Прайсинг через метод Фурье–COS*: характеристическая функция Rough Heston, дробное уравнение Риккати, GPU-реализация, сходимость по N факторам Бернштейна.

Глава 3. *Суррогатная модель FNO*: архитектура операторной сети, FiLM-кондиционирование, регуляризация на отсутствие арбитража, обучение.

Глава 4. *Калибровка*: анализ идентифицируемости, FIM, 3D-репараметризация, метод Ньютона–Гаусса с `jacfwd`.

Глава 5. *Эксперименты*: точность прайсера, качество FNO, сравнение Newton vs L-BFGS, потоковая калибровка, хеджирование, обучаемый показатель Хёрста.

Заключение.

Выводы и перспективы.

1 Обзор литературы: Смена парадигмы в количественных финансах

Классическая теория оценки производных финансовых инструментов (деривативов), фундаментально обоснованная работами Блэка, Шоулза и Мертона (1973) [1], опирается на стохастическое исчисление и решение уравнений в частных производных (PDE). Традиционные численные методы, такие как метод Монте-Карло (Monte Carlo) и конечно-разностные методы (Finite Difference Methods, FDM), являются «золотым стандартом» для задач низкой размерности. Однако современная финансовая индустрия сталкивается с вызовами, которые делают классические подходы вычислительно неэффективными:

1. **«Проклятие размерности» (Curse of Dimensionality):** Сложность сеточных методов растет экспоненциально ($O(n^d)$). Оценка опциона на корзину из 50–100 акций или расчет CVA (Credit Valuation Adjustment) для портфеля банка классическими методами требует неприемлемых временных затрат.
2. **Сложность калибровки (Calibration Bottleneck):** Современные модели, такие как модели грубой волатильности (Rough Volatility), лучше описывают рынок, но не имеют аналитических решений. Их калибровка через Монте-Карло занимает часы, что делает их неприменимыми для реал-тайм трейдинга.
3. **Рыночные трения:** Классические модели предполагают идеальный рынок. Учет транзакционных издержек и влияния на рынок (market impact) требует перехода к задачам стохастического управления, которые крайне сложны для аналитического решения.

В ответ на эти вызовы сформировалось направление *Scientific Machine Learning*, предлагающее использование глубоких нейронных сетей (DNN) как универсальных аппроксиматоров для решения финансовых задач.

1.1 Ведущие исследовательские группы и их вклад

Анализ современной литературы позволяет выделить несколько ключевых центров компетенций, каждый из которых специализируется на своем классе задач.

1.1.1 Группа Deep BSDE (Принстон, ETH Zurich)

Ключевые исследователи: Weinan E, Jiequn Han, Arnulf Jentzen.

- **Направление исследований:** Решение высокоразмерных параболических PDE (уравнение теплопроводности, Блэка-Шоулза, Аллена-Кана) с помощью нейронных сетей.
- **Достижения:**

- Разработали метод *Deep BSDE* [6], основанный на стохастической формулировке уравнений (Backward Stochastic Differential Equations).
- Впервые успешно решили нелинейное уравнение Блэка-Шоулза с риском дефолта для размерности $d = 100$ за разумное время.
- Доказали (теоретически и эмпирически), что нейросети могут преодолевать проклятие размерности, аппроксимируя градиент решения без построения пространственной сетки.
- **Оборудование:** В оригинальной статье использовались кластеры с GPU (NVIDIA Tesla K80/P100). Обучение модели для $d = 100$ занимало порядка 30–40 минут.
- **Open Source / Код:**
 - Официальный репозиторий: <https://github.com/frankhan91/DeepBSDE> (TensorFlow).
 - Существует множество современных портов на PyTorch.
- **Ключевые публикации:** [6], [7].

1.1.2 Группа DGM (University of Oxford, UIUC)

Ключевые исследователи: Justin Sirignano, Konstantinos Spiliopoulos.

- **Направление исследований:** Бессеточные методы (Meshfree methods) для решения PDE со свободной границей (Free Boundary Problems), что критически важно для оценки Американских опционов.
- **Достижения:**
 - Разработали метод *DGM (Deep Galerkin Method)* [8], который минимизирует невязку дифференциального оператора в случайно выбранных точках пространства.
 - Успешно решили задачу прайсинга Американских опционов в размерности до $d = 200$, что невозможно для традиционных методов.
 - Предложили метод Монте-Карло для оценки вторых производных (Гессиана), снижающий вычислительную сложность с $O(d^2)$ до $O(d)$.
- **Оборудование:** Использовали суперкомпьютер *Blue Waters* (NCSA) с тысячами узлов GPU, так как метод DGM требует значительных вычислительных ресурсов для сходимости в высоких размерностях.
- **Open Source / Код:**
 - Реализации доступны в библиотеке *DeepXDE*: <https://github.com/lululxvi/deepxde>.
- **Ключевые публикации:** [8].

1.1.3 Группа Deep Hedging (JP Morgan, ETH Zurich)

Ключевые исследователи: Hans Buehler, Lukas Gonon, Josef Teichmann, Ben Wood.

- **Направление исследований:** Прямая оптимизация торговых стратегий хеджирования в условиях рыночных трений (транзакционные издержки, ликвидность).
- **Достижения:**
 - Сформулировали задачу хеджирования не как поиск «справедливой цены», а как минимизацию выпуклой меры риска (Convex Risk Measure) [9].
 - Показали, что нейросети (LSTM/RNN) способны самостоятельно выучивать сложные стратегии (например, «wait-and-see» или бандинг), которые экономически эффективнее классической Дельта-стратегии при наличии комиссий.
 - Внедрили данную технологию в реальные бизнес-процессы банка JP Morgan.
- **Оборудование:** Для экспериментов использовались стандартные GPU (например, NVIDIA Tesla V100). Обучение на исторических данных занимает часы, но инференс происходит мгновенно.
- **Open Source / Код:**
 - Библиотека PFNet: <https://github.com/pfnet-research/deep-hedging>.
- **Ключевые публикации:** [9], [10] (теоретическое обоснование).

1.1.4 Группа Deep Calibration (King's College London, Imperial College)

Ключевые исследователи: Blanka Horvath, Christian Bayer, Aitor Muguruza.

- **Направление исследований:** Ускорение калибровки сложных стохастических моделей, в частности моделей грубой волатильности (Rough Volatility models: rBergomi, rHeston).
- **Достижения:**
 - Предложили двухэтапный подход: обучение нейросети-суррогата на синтетических данных (offline) и использование её для мгновенной калибровки (online) [11, 12].
 - Достигли ускорения процесса калибровки в 10 000 – 60 000 раз по сравнению с традиционными методами, сохраняя точность на уровне метода Монте-Карло.
- **Оборудование:** Обучение проводилось на стандартных GPU (Google Colab, NVIDIA GTX 1080). Генерация обучающей выборки является самой ресурсоемкой частью (требует CPU-кластера или времени), само обучение сети происходит быстро.

- **Open Source / Код:**

- Репозиторий авторов: <https://github.com/amuguruza/NN-StochVol-Calibrations>.
Содержит готовые ноутбуки для моделей Heston и Rough Bergomi.

- **Ключевые публикации:** [11], [12].

1.1.5 Группа прикладных исследований (TU Delft, CWI, NYU)

Ключевые исследователи: Cornelis Oosterlee, Shuaiqiang Liu (CaNN); Andrey Itkin.

- **Направление исследований:** Инженерная оптимизация нейросетевых солверов, обеспечение отсутствия арбитража, альтернативные методы калибровки.

- **Достижения:**

- Разработали фреймворк *CaNN (Calibration Neural Network)* [13], где калибровка параметров модели выполняется через обратное распространение ошибки (Backpropagation) на входной слой сети, без внешнего оптимизатора.
- Andrey Itkin предложил методы регуляризации (Soft Constraints) для устранения статического арбитража в предсказаниях нейросети и обосновал необходимость использования гладких функций активации (Softplus, ELU) для корректного расчета «Греков» [14].

- **Open Source / Код:**

- Примеры реализации CaNN доступны в сообществе (поиск по ключевым словам *Neural Network Calibration*).

- **Ключевые публикации:** [13], [14].

2 Архитектуры нейронных сетей в финансовом моделировании

Выбор архитектуры нейронной сети является критическим этапом в задачах количественных финансов. В отличие от задач компьютерного зрения или NLP, где существуют устоявшиеся стандарты (CNN, Transformers), в финансовом моделировании архитектура диктуется математической структурой задачи: марковостью процесса, зависимостью от траектории (path-dependency) и требованиями к гладкости производных.

Ниже приведен детальный анализ трех основных классов архитектур, применяемых в прайсинге и хеджировании.

2.1 Полносвязные сети (Feed-Forward Networks / MLP)

2.1.1 Теоретическая справка

Полносвязная сеть (Multilayer Perceptron, MLP) — это базовая архитектура, где каждый нейрон слоя l соединен со всеми нейронами слоя $l + 1$. Математически выход слоя описывается как:

$$h^{(l+1)} = \sigma(W^{(l)}h^{(l)} + b^{(l)}), \quad (1)$$

где W — матрица весов, b — вектор смещения, σ — нелинейная функция активации. Теоретическим обоснованием использования MLP служит *Универсальная теорема аппроксимации* (Hornik et al., 1989), утверждающая, что MLP с одним скрытым слоем может аппроксимировать любую непрерывную функцию с заданной точностью.

2.1.2 Применение в финансах

- **Deep Calibration:** Используется как суррогатная модель для отображения «Параметры модели $\rightarrow$ Цена опциона» или «Параметры $\rightarrow$ Поверхность волатильности» [11].
- **Inverse Problems:** Решение обратных задач, где по цене нужно восстановить параметры (например, локальную волатильность).

2.1.3 Преимущества и недостатки

Плюсы: Простота реализации; высокая скорость инференса; хорошо изученные методы регуляризации.

Минусы: Плохо работают с временными рядами и зависимостями от траектории; подвержены затуханию градиента при большом количестве слоев.

2.2 Остаточные сети (Residual Networks, ResNets)

2.2.1 Теоретическая справка

ResNets решают проблему затухания градиента в глубоких сетях путем введения «скип-соединений» (skip connections). Вместо того чтобы учить полное отображение $H(x)$, сеть учит остаточную функцию $F(x) = H(x) - x$. Слой ResNet выглядит так:

$$x_{l+1} = \sigma(W_l x_l + b_l) + x_l \quad (2)$$

Это позволяет сигналу беспрепятственно проходить через сотни слоев, что критически важно для аппроксимации сложных функционалов.

2.2.2 Применение в финансах

- **Решение PDE (DGM, Deep BSDE):** Методы DGM [8] и Deep BSDE [6] требуют очень глубоких сетей (до 50 слоев) для точной аппроксимации градиента решения в высокой размерности. Без ResNet-блоков такие сети практически невозможно обучить.

2.2.3 Преимущества и недостатки

Плюсы: Позволяют строить очень глубокие модели; эффективно обучаются; обеспечивают высокую точность аппроксимации сложных поверхностей.

Минусы: Сложнее в настройке гиперпараметров; требуют больше памяти при обучении по сравнению с простым MLP.

2.3 Рекуррентные сети (RNN / LSTM)

2.3.1 Теоретическая справка

Рекуррентные сети предназначены для обработки последовательностей. В отличие от FNN, они имеют «внутреннюю память» (hidden state h_t), которая передается от шага к шагу:

$$h_t = \sigma(W_x x_t + W_h h_{t-1} + b). \quad (3)$$

Архитектура **LSTM** (Long Short-Term Memory) добавляет механизм «ворот» (gates), позволяющий сети учить долгосрочные зависимости и избегать взрыва градиента.

2.3.2 Применение в финансах

- **Deep Hedging:** В задачах с транзакционными издержками оптимальное действие сегодня зависит от портфеля вчера. Это классическая марковская зависимость, идеально моделируемая LSTM [9].
- **Path-dependent опционы:** Оценка Азиатских опционов или Lookback-опционов, где выплата зависит от всей траектории цены.
- **Прогнозирование временных рядов:** Предсказание реализованной волатильности.

2.3.3 Преимущества и недостатки

Плюсы: Естественным образом работают с временными рядами; учитывают историю процесса (memory effect); необходимы для моделей с трениями.

Минусы: Медленное обучение (плохо распараллеливаются во времени); сложны в интерпретации; требуют больших вычислительных ресурсов (VRAM).

2.4 Сравнительный анализ архитектур

В Таблице 1 представлено сравнение архитектур в контексте задач дипломной работы.

Table 1: Сравнение архитектур нейронных сетей для финансового моделирования

Архитектура	Ключевая особенность	Область применения	Сложность обучения
FNN (MLP)	Универсальная аппроксимация, простота	Deep Calibration, простые суррогаты	Низкая (CPU/GPU)
ResNet	Skip connections, большая глубина	Решение PDE (DGM, Deep BSDE), высокая размерность	Средняя/Высокая (GPU)
LSTM/GRU	Память, обработка последовательностей	Deep Hedging, Path-dependent опционы	Высокая (требует мощных GPU)

Вывод: Для задач калибровки (Deep Calibration), где требуется мгновенное отображение параметров в цену, оптимальным выбором является FNN. Однако для задач хеджирования (Deep Hedging) и решения PDE в высокой размерности необходимо использование более сложных архитектур типа LSTM и ResNet соответственно, так как они способны улавливать временные зависимости и сложную геометрию решений.

3 Детальный анализ ключевых исследований

В данном разделе проводится глубокий анализ фундаментальных работ, формирующих методологическую базу исследования.

3.1 Метод Deep BSDE: Стохастическая оптимизация для высокоразмерных PDE

Статья: *Solving high-dimensional partial differential equations using deep learning* [6].

3.1.1 Проблема и мотивация

Классические сеточные методы (Finite Difference Methods) страдают от «проклятия размерности»: количество узлов сетки растет как N^d , где d — количество активов. Это делает невозможным их применение для $d > 5$. С другой стороны, стандартный метод Монте-Карло, хотя и работает в высоких размерностях, применим только для *линейных* PDE (через формулу Фейнмана-Каца).

Однако многие современные финансовые задачи (например, оценка CVA, прайсинг с учетом дефолтного риска или различных ставок заимствования) описываются **нелинейными** PDE. Метод Deep BSDE, предложенный группой исследователей из Принстона и ETH Zurich (Weinan E, J. Han, A. Jentzen), стал первым алгоритмом, позволившим решать такие уравнения в размерности $d = 100$ за полиномиальное время.

3.1.2 Математическая теория

Метод базируется на теории обратных стохастических дифференциальных уравнений (Backward Stochastic Differential Equations, BSDE). Рассмотрим полулинейное параболическое уравнение для функции цены $u(t, x)$:

$$\frac{\partial u}{\partial t} + \frac{1}{2} \text{Tr}(\sigma \sigma^T \text{Hess}_x u) + \nabla u \cdot \mu + f(t, x, u, \sigma^T \nabla u) = 0, \quad (4)$$

с терминальным условием $u(T, x) = g(x)$. Здесь $f(\cdot)$ — нелинейная функция, описывающая, например, разницу процентных ставок или риск контрагента.

Согласно нелинейной версии формулы Фейнмана-Каца (теорема Парду-Пенга), решение этого PDE эквивалентно решению системы стохастических уравнений:

1. Прямое уравнение (Forward SDE) — Динамика активов:

$$X_t = \xi + \int_0^t \mu(s, X_s) ds + \int_0^t \sigma(s, X_s) dW_s \quad (5)$$

2. Обратное уравнение (Backward SDE) — Динамика портфеля:

$$Y_t = g(X_T) - \int_t^T f(s, X_s, Y_s, Z_s) ds - \int_t^T Z_s^T dW_s \quad (6)$$

Финансовый смысл переменных:

- $Y_t = u(t, X_t)$ — цена дериватива (или стоимость портфеля) в момент t .
- $Z_t = \sigma^T(t, X_t) \nabla u(t, X_t)$ — вектор хеджирования (аналог «Дельты», умноженной на волатильность), показывающий, сколько риска нужно перекрыть.

3.1.3 Алгоритм и применение нейросетей

В классических численных методах Z_t неизвестно. Идея Deep BSDE заключается в аппроксимации неизвестной функции градиента Z_t нейронной сетью на каждом временном шаге.

Алгоритм дискретизируется по времени $0 = t_0 < t_1 < \dots < t_N = T$ (схема Эйлера-Маруямы):

$$Y_{n+1} \approx Y_n - f(t_n, X_n, Y_n, Z_n) \Delta t + Z_n^T \Delta W_n \quad (7)$$

Здесь $Z_n = \text{NeuralNet}(X_n; \theta_n)$. Процесс начинается с Y_0 (искомая цена, обучаемый параметр) и идет вперед во времени.

Функция потерь (Loss Function) строится на невязке терминального условия:

$$L(\theta) = \mathbb{E} [|Y_T(\theta) - g(X_T)|^2] \quad (8)$$

Минимизируя разницу между тем, что мы «наsimулировали» (Y_T), и тем, что должны выплатить по контракту ($g(X_T)$), сеть одновременно находит справедливую цену Y_0 и стратегию хеджирования $\{Z_t\}$.

3.1.4 Область применения и преимущества

- **Где используется:** Основная ниша метода — задачи оценки кредитных рисков (XVA: CVA/DVA/FVA) для крупных портфелей банков, а также прайсинг опционов на корзины акций (Basket Options) в условиях нелинейной волатильности или ставок.
- **Кем используется:** Исследовательские подразделения (Quant Research) крупных инвестиционных банков (JP Morgan, Goldman Sachs) и хедж-фонды, работающие со сложными структурными продуктами.
- **Когда применять в дипломе:**
 - Если размерность задачи $d \geq 10$ (классические сетки умирают).
 - Если задача нелинейная (обычный Монте-Карло не подходит).
 - Если нужно найти не только цену, но и хеджирующую стратегию.
- **Ограничения:** В отличие от метода DGM, Deep BSDE находит решение только для *одного* фиксированного начального условия ($t = 0, X_0$). Если цена актива X_0 изменится, сеть нужно переобучать заново (или использовать методы параметризации X_0 , что сложнее).

3.2 Метод DGM: Глобальное решение и задачи со свободной границей

Статья: *DGM: A deep learning algorithm for solving partial differential equations* [8].

3.2.1 Проблема и мотивация

В то время как метод Deep BSDE решает уравнение вдоль конкретных стохастических траекторий (Lagrangian approach), он имеет существенный недостаток: решение находится только для одной фиксированной точки ($t = 0, X_0$). Если рыночная цена актива изменится, процедуру обучения нужно запускать заново.

Метод DGM (Deep Galerkin Method) предлагает *глобальный* подход (Eulerian approach). Цель метода — обучить нейросеть, которая будет аппроксимировать функцию цены $u(t, x)$ во всей области определения сразу. Это позволяет не только получать мгновенные оценки при изменении цены актива, но и эффективно решать задачи с **свободной границей** (Free Boundary Problems), к которым относится оценка Американских опционов — наиболее распространенного типа деривативов на биржевых рынках.

3.2.2 Математическая теория

Рассмотрим общее параболическое PDE для функции $u(t, x)$ в области $\Omega \subset \mathbb{R}^d$ на интервале $[0, T]$:

$$(\partial_t + \mathcal{L})u(t, x) = 0, \quad (t, x) \in [0, T] \times \Omega, \quad (9)$$

где $\mathcal{L}$ — дифференциальный оператор (для финансов — оператор Блэка-Шоулза или Хестона), включающий вторые производные (Гессиан). Также заданы граничные условия на $\partial\Omega$ и терминальное условие при $t = T$.

Вместо построения сетки, авторы предлагают искать решение в виде нейронной сети $f(t, x; \theta)$. Функция потерь конструируется как взвешенная сумма квадратов невязок (residuals) уравнения в случайно выбранных точках:

$$J(f) = \underbrace{\|\partial_t f + \mathcal{L}f\|_{[0, T] \times \Omega}^2}_{\text{Ошибка внутри области}} + \underbrace{\|f - g\|_{\partial\Omega \times [0, T]}^2}_{\text{Граничные условия}} + \underbrace{\|f(T, \cdot) - \text{Payoff}\|_{\Omega}^2}_{\text{Терминальное условие}} \quad (10)$$

Ключевая математическая инновация: Вычисление оператора $\mathcal{L}f$ требует нахождения вторых производных (матрицы Гессиана). Для нейросети в высокой размерности d это операция сложности $O(d^2)$, что вычислительно запретительно. Авторы предложили метод несмещенной оценки вторых производных с помощью Монте-Карло (аналог Hutchinson's trace estimator), что снижает сложность до $O(d)$ и позволяет работать с размерностями $d = 200$.

3.2.3 Алгоритм и Архитектура

Процесс обучения (Training Loop) выглядит следующим образом:

1. На каждой итерации генерируется пакет (batch) случайных точек внутри области, на границах и во времени T .
2. С помощью автоматического дифференцирования (Autograd) вычисляются производные нейросети по входам $\frac{\partial f}{\partial t}, \frac{\partial f}{\partial x}, \frac{\partial^2 f}{\partial x^2}$.
3. Вычисляется невязка уравнения и обновляются веса θ методом стохастического градиентного спуска (Adam).

Архитектура сети, предложенная авторами, вдохновлена LSTM и ResNet (DGM-Layer). Она включает проброс связей (skip-connections) и мультипликативные взаимодействия, что позволяет избежать затухания градиента при вычислении производных высоких порядков.

3.2.4 Область применения и преимущества

- **Американские опционы:** Это главная "фишка" метода. Задача прайсинга Американского опциона формулируется как задача с препятствием (obstacle problem) или задача со свободной границей. В DGM это реализуется добавлением

штрафа в Loss-функцию, который гарантирует, что цена опциона всегда не ниже выплаты при досрочном исполнении. Deep BSDE справляется с этим значительно хуже (требует Reflected BSDE).

- **Универсальность:** Одна обученная модель дает поверхность цен для любых S_t и t .
- **Сложность вычислений:** Метод является вычислительно тяжелым («expensive»). Взятие вторых производных через граф вычислений нейросети требует большого объема видеопамяти (VRAM) и мощных GPU.
- **Когда применять в дипломе:**
 - Если тема касается **Американских опционов** или Бермудских опционов.
 - Если нужно исследовать зависимость цены от состояния рынка (построить график Value Surface).
 - Если имеется доступ к хорошим вычислительным ресурсам (NVIDIA Tesla T4/P100 и выше).

4 Детальный анализ ключевых исследований

В данном разделе проводится анализ фундаментальных работ, формирующих методологическую базу применения глубокого обучения в задачах финансовой математики. Рассматриваются методы решения уравнений в частных производных (PDE), подходы к хеджированию в условиях рыночных несовершенств, а также алгоритмы калибровки моделей.

4.1 Метод Deep BSDE: Стохастическая оптимизация для высокоразмерных PDE

Статья: *Solving high-dimensional partial differential equations using deep learning* [6].

4.1.1 Проблема и мотивация

Классические сеточные методы (Finite Difference Methods) подвержены «проклятию размерности»: количество узлов сетки растет экспоненциально с увеличением числа активов, что делает их неприменимыми для размерностей $d > 5$. Стандартный метод Монте-Карло, работающий в высоких размерностях, применим преимущественно для линейных PDE. Однако многие современные финансовые задачи (например, оценка CVA, прайсинг с учетом дефолтного риска или различных ставок заимствования) описываются **нелинейными** PDE. Метод Deep BSDE стал первым алгоритмом, позволившим решать такие уравнения в размерности $d = 100$ за полиномиальное время.

4.1.2 Математическая теория

Метод базируется на теории обратных стохастических дифференциальных уравнений (Backward Stochastic Differential Equations, BSDE). Рассмотрим полулинейное параболическое уравнение для функции цены $u(t, x)$:

$$\frac{\partial u}{\partial t} + \frac{1}{2} \text{Tr}(\sigma \sigma^T \text{Hess}_x u) + \nabla u \cdot \mu + f(t, x, u, \sigma^T \nabla u) = 0, \quad (11)$$

с терминальным условием $u(T, x) = g(x)$.

Согласно нелинейной версии формулы Фейнмана-Каца, решение этого PDE эквивалентно решению системы стохастических уравнений:

$$Y_t = g(X_T) - \int_t^T f(s, X_s, Y_s, Z_s) ds - \int_t^T Z_s^T dW_s, \quad (12)$$

где $Y_t = u(t, X_t)$ — цена дериватива, а $Z_t = \sigma^T \nabla u(t, X_t)$ — вектор хеджирования.

4.1.3 Алгоритм и применимость

Идея метода заключается в аппроксимации неизвестной функции градиента Z_t нейронной сетью на каждом временном шаге дискретизации Эйлера-Маруямы. Функция потерь строится на невязке терминального условия:

$$L(\theta) = \mathbb{E} [|Y_T(\theta) - g(X_T)|^2]. \quad (13)$$

Данный подход является стандартом (SOTA) для задач высокой размерности (например, оценка корзинных опционов). Его применение целесообразно в случаях, когда требуется найти не только цену, но и стратегию хеджирования в условиях нелинейной динамики.

4.2 DGM: Бессеточный метод Галёркина

Статья: *DGM: A deep learning algorithm for solving partial differential equations* [8].

4.2.1 Проблема и мотивация

Метод Deep BSDE решает уравнение вдоль конкретных стохастических траекторий, что ограничивает решение одной фиксированной точкой ($t = 0, X_0$). Метод DGM (Deep Galerkin Method) предлагает глобальный подход, обучая нейросеть аппроксимировать функцию цены во всей области определения. Это позволяет эффективно решать задачи со свободной границей (Free Boundary Problems), к которым относится оценка Американских опционов.

4.2.2 Математическая теория

Обучение сводится к минимизации квадрата невязки дифференциального оператора $\mathcal{L}$ в случайно выбранных точках области Ω :

$$J(f) = \|\partial_t f + \mathcal{L}f\|_{\Omega}^2 + \|f - g\|_{\partial\Omega}^2 + \|f(T, \cdot) - \text{Payoff}\|^2. \quad (14)$$

Авторы предложили метод несмещенной оценки вторых производных (Гессиана) с помощью Монте-Карло, что снижает вычислительную сложность с $O(d^2)$ до $O(d)$.

4.2.3 Алгоритм и применимость

Архитектура сети, основанная на DGM-слоях (модификация ResNet/LSTM), позволяет избежать затухания градиента при вычислении производных высоких порядков. Метод наиболее эффективен для задач, требующих построения полной поверхности цен (Value Surface) и оценки опционов с возможностью досрочного исполнения, однако требует значительных вычислительных ресурсов (GPU) для обучения.

4.3 Deep Hedging: Оптимизация торговых стратегий при наличии трений

Статья: *Deep Hedging* [9].

4.3.1 Проблема и мотивация

Классическая теория (Блэк-Шоулз) предполагает отсутствие транзакционных издержек и возможность непрерывной торговли. В реальных условиях эти допущения нарушаются, делая классическое дельта-хеджирование субоптимальным. Подход *Deep Hedging* предлагает смену парадигмы: от поиска «теоретической цены» к поиску оптимальной стратегии, минимизирующей риск с учетом рыночных трений.

4.3.2 Математическая теория

Задача формулируется как проблема стохастического управления. Итоговый капитал PL_T зависит от стратегии δ и транзакционных издержек C_k , которые зависят от оборота $|\delta_k - \delta_{k-1}|$:

$$PL_T(\delta) = -Z + p_0 + \sum (\delta \cdot \Delta S) - \sum C_k(\delta_k, \delta_{k-1}). \quad (15)$$

Целью является минимизация выпуклой меры риска ρ (например, CVaR или экспоненциальной полезности):

$$\pi(Z) = \inf_{\delta \in \mathcal{H}} \rho(-PL_T(\delta)). \quad (16)$$

4.3.3 Алгоритм и применимость

Поскольку издержки зависят от предыдущего состояния портфеля, задача является немарковской относительно цены актива. Для решения используются рекуррентные нейронные сети (RNN/LSTM). Метод демонстрирует высокую практическую ценность, позволяя моделировать стратегии, превосходящие классические подходы по показателю риск/доходность на рынках с комиссиями.

4.4 Deep Calibration: Ускорение калибровки сложных моделей

Статья: *Deep learning volatility: a deep neural network perspective on pricing and calibration in (rough) volatility models* [11].

4.4.1 Проблема и мотивация

Современные модели, такие как *Rough Volatility*, точно описывают динамику рынка, но не имеют аналитических решений. Их калибровка традиционными методами требует множественных симуляций Монте-Карло, что занимает неприемлемо много времени для операционной деятельности.

4.4.2 Суть метода

Предлагается двухэтапный подход:

1. **Офлайн-обучение:** Нейросеть обучается на синтетических данных аппроксимировать функцию цены $F(\theta) \approx P(\theta)$. Сеть выступает в роли детерминированного суррогата стохастической модели.
2. **Онлайн-калибровка:** При поступлении рыночных данных параметры модели θ^* находятся путем минимизации разницы между предсказанием сети и рынком с использованием градиентного оптимизатора.

4.4.3 Алгоритм и применимость

Использование суррогатной модели позволяет ускорить процесс калибровки в 10 000–60 000 раз. Метод применим к широкому классу моделей (Хестон, Rough Bergomi) и является эффективным решением проблемы «вычислительного бутылочного горлышка» в финансовом моделировании.

4.5 CaNN: Калибровка через механизм обратного распространения ошибки

Статья: *A neural network-based framework for financial model calibration* [13].

4.5.1 Суть метода

В отличие от подхода с внешним оптимизатором, фреймворк CaNN (Calibration Neural Network) использует механизм обратного распространения ошибки (Backpropagation) для решения обратной задачи. Процесс разделен на две фазы: обучение сети (forward pass) и калибровка (backward pass). На этапе калибровки веса сети замораживаются, а входной слой, соответствующий параметрам модели, объявляется обучаемым. Градиент ошибки между предсказанной и рыночной ценой распространяется до входа, обновляя параметры модели.

4.5.2 Применимость

Данный подход позволяет выполнять калибровку, используя встроенные возможности фреймворков глубокого обучения (например, PyTorch) без подключения сторонних солверов, что упрощает архитектуру программного решения.

4.6 Теоретическое обоснование: Рандомизированные сигнатуры

Статья: *Discrete-time signatures and randomness in reservoir computing* [10].

4.6.1 Суть метода

Статья предоставляет фундаментальное теоретическое обоснование эффективности рекуррентных нейронных сетей (RNN) в задачах прогнозирования финансовых временных рядов. Авторы связывают RNN с теорией сигнатур путей (Path Signatures) и рядами Вольтерра, доказывая свойство универсальной аппроксимации для фильтров с затухающей памятью.

4.6.2 Применимость

Работа обосновывает выбор архитектур типа LSTM/GRU для задач, зависящих от траектории (path-dependent), и предлагает альтернативные методы на основе Reservoir Computing, которые могут быть вычислительно эффективнее полного обучения глубоких сетей.

4.7 Инженерные аспекты: Обеспечение финансовой корректности

Статья: *Deep learning calibration of option pricing models: some pitfalls and solutions* [14].

4.7.1 Проблема и решения

Прямое применение стандартных методов ML может приводить к моделям, допускающим статический арбитраж или имеющим разрывные производные (Греки). Автор предлагает ряд инженерных решений:

1. Использование гладких функций активации класса C^2 (Softplus, ELU) вместо ReLU для корректного вычисления вторых производных (Гамма).
2. Введение штрафов (Soft Constraints) в функцию потерь за нарушение условий отсутствия арбитража (например, нарушение монотонности цены по времени или выпуклости по страйку).

4.7.2 Применимость

Данные рекомендации являются критически важными при практической реализации моделей для обеспечения их робастности и соответствия финансовой теории.

5 Сравнительный анализ и Выводы

Table 2: Сравнение нейросетевых подходов

Метод	Задача	Сложность	Ресурсы (GPU)
Deep BSDE	Прайсинг, $d \sim 100$	Средняя	Средние (T4)
DGM	Американские опционы	Высокая	Высокие (V100/A100)
Deep Hedging	Хеджирование с трениями	Средняя	Средние/Высокие
Deep Calibration	Мгновенная калибровка	Низкая	Низкие (любой GPU)

5.1 План практического исследования

На основе проведенного обзора, для практической части дипломной работы выбран метод **Deep Calibration**. Данный подход позволяет решить актуальную проблему медленной калибровки, демонстрируя превосходство над классическими методами. Для реализации будет использован фреймворк PyTorch, а обучение будет проводиться с использованием облачных ресурсов (Google Colab/Kaggle) для ускорения вычислений.

6 Математические основы

6.1 Рынок опционов и подразумеваемая волатильность

Рассмотрим финансовый рынок с вероятностным пространством $(\Omega, \mathcal{F}, (\mathcal{F}_t)_{t \geq 0}, \mathbb{Q})$, где $\mathbb{Q}$ — мартингальная мера. Цена базового актива (например, S&P 500) описывается процессом $(S_t)_{t \geq 0}$.

Определение 6.1 (Европейский колл-опцион). *Европейский колл-опцион с ценой исполнения K и сроком погашения T даёт держателю право (но не обязательство) купить базовый актив за K в момент T . Его справедливая цена в момент $t = 0$ при нулевой безрисковой ставке:*

$$C(S_0, K, T) = \mathbb{E}^{\mathbb{Q}}[(S_T - K)^+].$$

Определение 6.2 (Подразумеваемая волатильность). *Подразумеваемая волатильность $IV(K, T)$ — это такое $\sigma > 0$, при котором формула Блэка–Шоулса воспроизводит наблюдаемую цену:*

$$C_{BS}(S_0, K, T, \sigma) = C^{\text{market}}(K, T).$$

Как функция (K, T) , IV образует поверхность подразумеваемой волатильности.

Поверхность IV нетривиальна: для каждой даты погашения T она образует «улыбку» или «ухмылку» (volatility smile/skew) по страйку K . Модель Блэка–Шоулса предсказывает плоскую поверхность, что противоречит рыночным наблюдениям.

6.2 Дробное броуновское движение и грубая волатильность

Определение 6.3 (Дробное броуновское движение). *Дробное броуновское движение (fractional Brownian motion, fBm) $B^H = (B_t^H)_{t \geq 0}$ с показателем Хёрста $H \in (0, 1)$ — это центрированный гауссовский процесс с ковариацией:*

$$\mathbb{E}[B_s^H B_t^H] = \frac{1}{2}(s^{2H} + t^{2H} - |t - s|^{2H}).$$

При $H = 1/2$ это стандартное броуновское движение; при $H < 1/2$ — грубый (rough) процесс с траекториями более нерегулярными, чем BM .

Empirically, Gatheral, Jaisson & Rosenbaum (2018) [4] показали, что показатель Хёрста реализованной волатильности S&P 500 оценивается как $\hat{H} \approx 0,08$ — значительно ниже $1/2$, что мотивирует использование $H < 1/2$ в моделях.

Замечание 6.1. При $H < 1/2$ фБД не является ни марковским, ни полумартингалом. Стохастическое исчисление для него требует специального обращения (исчисление Молчанова–Нуалара или репрезентация через ядро Вольтерра).

6.3 Модель Rough Heston

Определение 6.4 (Rough Heston [5]). В риск-нейтральной мере цена актива S_t и мгновенная дисперсия V_t удовлетворяют:

$$\frac{dS_t}{S_t} = \sqrt{V_t} dW_t^S, \quad (17)$$

$$V_t = V_0 + \frac{1}{\Gamma(H + \frac{1}{2})} \int_0^t (t-s)^{H-1/2} \kappa(\theta - V_s) ds + \frac{\sigma}{\Gamma(H + \frac{1}{2})} \int_0^t (t-s)^{H-1/2} \sqrt{V_s} dW_t^V, \quad (18)$$

где W^S и W^V — броуновские движения с корреляцией $d\langle W^S, W^V \rangle_t = \rho dt$.

Параметры модели: $\theta = (\kappa, \theta_\infty, \sigma, \rho, V_0, H)$, где:

- $\kappa > 0$ — скорость возврата к среднему;
- $\theta_\infty > 0$ — долгосрочный уровень дисперсии;
- $\sigma > 0$ — волатильность волатильности (vol-of-vol);
- $\rho \in (-1, 0)$ — корреляция (отрицательна для equity);
- $V_0 > 0$ — начальная мгновенная дисперсия;
- $H \in (0, 1/2)$ — показатель Хёрста.

При $H = 1/2$ модель вырождается в классическую модель Heston (1993) [2].

6.4 Проблема идентифицируемости

Шесть параметров модели нельзя восстановить независимо из типичного набора наблюдаемых опционных цен — система близка к *нерегулярной* (ill-posed). Это проявляется в малых собственных значениях матрицы Фишера (FIM, подробнее в Главе 4): число обусловленности $\kappa(\mathbf{I}_{6 \times 6}) \sim 10^6$, что делает калибровку крайне чувствительной к шуму.

Анализ чувствительности (Delta-IV, Appendix A) показывает, что из 6 параметров лишь **три** определяют форму поверхности IV на горизонтах $T \in [0, 1; 2, 0]$:

$$v_0 \equiv V_0 \quad (\text{текущая дисперсия}), \quad (19)$$

$$\zeta \equiv \sigma \rho \quad (\text{наклон улыбки}), \quad (20)$$

$$\lambda \equiv \sigma \sqrt{1 - \rho^2} \quad (\text{кривизна улыбки}). \quad (21)$$

Параметры κ, θ_∞, H фиксируются при значениях $\kappa^* = 1, 0$, $\theta^* = 0, 08$, $H^* = 0, 08$ на основе исторических оценок. В Главе 4 обоснована эта редукция через FIM: переход $6D \rightarrow 3D$ снижает число обусловленности в $1301 \times$.

6.5 Лифтинговое представление и дробное уравнение Риккати

Ключевое преимущество Rough Heston — наличие **полуаналитической характеристической функции**, доступной через решение дробного уравнения Риккати. El Euch & Rosenbaum (2019) показали, что:

$$\mathbb{E}^{\mathbb{Q}}[e^{iu \ln S_T}] = \exp\left(V_0 \Psi(u, T) + \kappa \theta_{\infty} \int_0^T \Psi(u, t) dt\right),$$

где $\Psi(u, \cdot)$ удовлетворяет дробному уравнению Риккати:

$$\Psi(u, t) = \frac{iu(iu - 1)}{2} + \int_0^t K(t - s) (\kappa - \sigma \rho iu) \Psi(u, s) ds - \frac{\sigma^2}{2} \int_0^t K(t - s) \Psi(u, s)^2 ds, \quad (22)$$

где ядро $K(t) = t^{H-1/2}/\Gamma(H + 1/2)$.

6.5.1 Приближение Бернштейна (лифтинг)

Уравнение (22) интегро-дифференциальное и не имеет замкнутого решения. Аппроксимируем ядро K суммой экспонент (приближение Бернштейна, El Euch & Rosenbaum, 2019):

$$K(t) \approx K^N(t) = \sum_{i=1}^N c_i e^{-x_i t}, \quad x_i = r_N^{i-1-N/2}, \quad c_i = \frac{x_i^{-(H+1/2)}}{\sum_j x_j^{-(H+1/2)}},$$

где $r_N = 1 + 10N^{-0.9}$. При $N = 40$ погрешность ядра:

$$\|K - K^{40}\|_{L^2[0, T]} \leq \varepsilon \approx 10 \text{ б.п. (IV)}.$$

Замена ядра превращает уравнение (22) в систему N ОДУ (*поднятая модель*, lifted Heston):

$$\frac{d\psi_i}{dt} = \frac{iu(iu - 1)}{2} c_i - (\kappa + x_i) \psi_i - \sigma \rho iu \sum_j \psi_j - \frac{\sigma^2}{2} \left(\sum_j \psi_j \right)^2 c_i, \quad i = 1, \dots, N. \quad (23)$$

Характеристическая функция:

$$\ln \varphi(u, T) = V_0 \sum_i \psi_i(T) + \kappa \theta_{\infty} \int_0^T \sum_i \psi_i(t) dt.$$

6.6 Условия отсутствия арбитража для поверхностей IV

Не всякая поверхность IV(K, T) соответствует отсутствию арбитража. Gatheral & Jacquier (2011) [15] дали необходимые и достаточные условия в терминах тотальной дисперсии $w(k) = \text{IV}^2(k, T) \cdot T$:

Теорема 6.1 (Условия ГДЖ). *Поверхность IV свободна от статического арбитража тогда и только тогда, когда выполнены:*

1. **Calendar spread:** $w(k, T_1) \leq w(k, T_2)$ при $T_1 < T_2$ (тотальная дисперсия не убывает по сроку).

2. **Butterfly:** $g(k) \geq 0$, где

$$g(k) = \left(1 - \frac{k w'(k)}{2w(k)}\right)^2 - \frac{(w'(k))^2}{4} \left(\frac{1}{w(k)} + \frac{1}{4}\right) + \frac{w''(k)}{2}.$$

Эти условия используются как **регуляризующий штраф** при обучении FNO (подробнее в Главе 3, уравнение (30)).

7 GPU-ускоренный прайсинг методом Фурье–COS

7.1 Метод Фурье–COS

Метод Фурье–COS (Fang & Oosterlee, 2008) [16] позволяет вычислять цены европейских опционов через характеристическую функцию. Для логарифма цены $x = \ln(S_T/S_0)$ плотность $f(x)$ аппроксимируется усечённым косинус-рядом на $[a, b]$:

$$f(x) \approx \frac{2}{b-a} \sum_{k=0}^{N_{\cos}-1} {}' \operatorname{Re} \left[\hat{\varphi} \left(\frac{k\pi}{b-a} \right) e^{ik\pi(a-a)/(b-a)} \right] \cos \left(\frac{k\pi(x-a)}{b-a} \right),$$

где $\hat{\varphi}(u) = \varphi(u) e^{-iua}$ и $'$ означает деление первого слагаемого на 2. Цена колл-опциона:

$$C(K, T) = S_0 e^{-rT} \frac{2}{b-a} \sum_{k=0}^{N_{\cos}-1} {}' \operatorname{Re} \left[\varphi \left(\frac{k\pi}{b-a} \right) e^{-ik\pi a/(b-a)} \right] V_k, \quad (24)$$

где *коэффициенты выплат* V_k вычисляются аналитически (Fang & Oosterlee, 2008).

Параметры метода:

- $[a, b]$ — усечение: выбрано $a = -0,5 \cdot \sqrt{T}$, $b = 0,5 \cdot \sqrt{T}$ (покрывает 5σ для $\sigma \leq 1$);
- $N_{\cos}$ — число слагаемых: выбрано 128 (анализ в §7.4).

7.2 Решение лифтинговой системы ОДУ

Для вычисления $\varphi(u, T)$ из (23) необходимо решить систему $N \times N_u$ ОДУ (где N — число факторов Бернштейна, N_u — число частот COS). Нивное применение метода Рунге–Кутты 4-го порядка (RK4) неустойчиво при больших x_i (жёсткая система): линейный вклад $-x_i \psi_i$ приводит к шагу устойчивости $\Delta t \leq 2/x_i$, тогда как $x_{40} \approx 6275$ требует $\Delta t \leq 3 \times 10^{-4}$ — неприемлемо мало.

7.2.1 Exponential Midpoint Integrator

Используем *экспоненциальный полушаговый интегратор* (exponential midpoint rule) [17]. Перепишем систему:

$$\frac{d\psi_i}{dt} = -\kappa x_i \psi_i + g_i(u, \psi),$$

где g_i включает нелинейные члены. Шаг интегратора:

$$\tilde{\psi}_i(t + h/2) = e^{-\kappa x_i h/2} \psi_i(t) + \frac{1 - e^{-\kappa x_i h/2}}{\kappa x_i} g_i(t, \psi(t)), \quad (25)$$

$$\psi_i(t + h) = e^{-\kappa x_i h} \psi_i(t) + \frac{1 - e^{-\kappa x_i h}}{\kappa x_i} g_i\left(t + \frac{h}{2}, \tilde{\psi}(t + h/2)\right). \quad (26)$$

Интегратор **безусловно устойчив** по линейному члену (точная экспоненциальная пропация) и является 2-го порядка по нелинейному члену. При $h = 1/200$ (200 шагов

на единицу времени) погрешность $IV \leq 2$ б.п. при $T = 0,1$ (Таблица 4).

7.3 GPU-реализация: пакетная обработка

Реализация написана на PyTorch (CUDA) и обрабатывает $B = 1024$ наборов параметров параллельно:

1. **Инициализация факторов** ($\mathcal{O}(B \cdot N)$): тензор весов $\mathbf{c} \in \mathbb{R}^{B \times N}$, узлы $\mathbf{x} \in \mathbb{R}^N$ (один на все образцы).
2. **ODE-шаги** ($\mathcal{O}(T_{\max}/h \cdot B \cdot N \cdot N_u)$): основная GPU-нагрузка; реализована в смешанной точности (complex64 для состояния ODE, complex128 для накопителя интеграла).
3. **COS-суммирование** ($\mathcal{O}(B \cdot N_T \cdot N_K \cdot N_u)$): обратное преобразование из φ в цены, затем перевод в IV через алгоритм Джэкэла (Jaeskel, 2015) [18].

Итоговый батч $B = 1024$, $N = 40$, $N_u = 128$, $N_T = 8$, $N_K = 11$ обрабатывается за 1,4 с на RTX 3060, то есть $\approx 1,4$ мс на один набор параметров.

7.4 Сходимость по числу факторов Бернштейна

Таблица 3 показывает максимальную ошибку IV (в базисных пунктах) относительно референса $N = 128$ при тестовых параметрах ($\kappa = 1, \theta = 0,08, \sigma = 0,5, \rho = -0,5, v_0 = 0,08, H = 0,08$).

Table 3: Сходимость по числу факторов Бернштейна N ($N_{\cos} = 128$)

N	$T = 0,1$ (б.п.)	$T = 1,0$ (б.п.)	Global Max (б.п.)	Время (с)
5	40,2	151,3	254,5	0,122
10	34,2	99,2	157,9	0,122
20	21,6	54,0	83,9	0,119
40	10,5	24,7	37,9	0,119
80	3,3	7,5	11,4	0,123
128	—	—	—	0,592 (эталон)

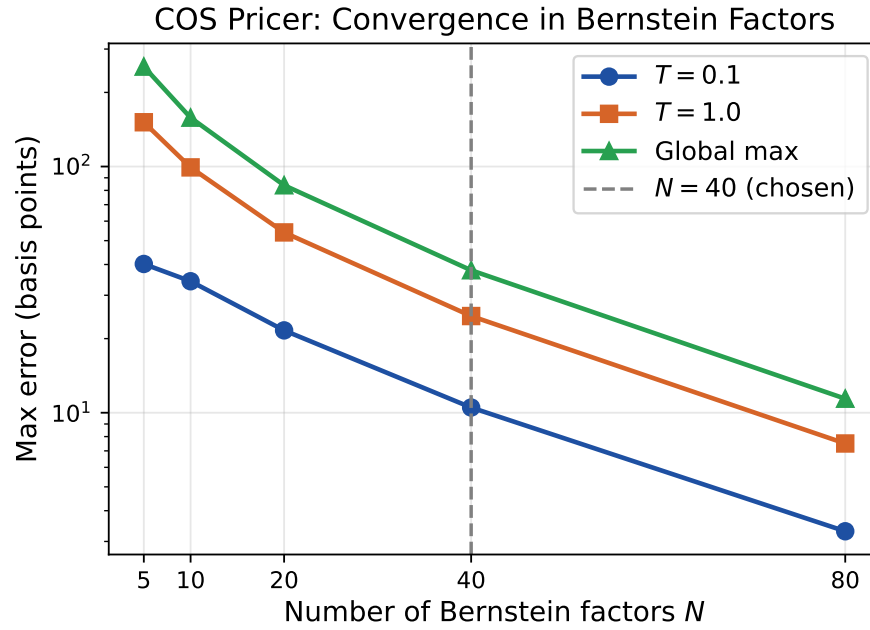


Figure 1: Сходимость прайсера Фурье–COS по числу факторов Бернштейна. Пунктиром отмечен выбранный $N = 40$.

Выбор $N = 40$: обеспечивает максимальную ошибку 37,9 б.п. при вычислительной стоимости, идентичной $N = 5$ (0,119с, поскольку операция ограничена памятью, а не арифметикой). Соотношение точность/скорость оптимально по сравнению с $N = 80$ (11 б.п., +3% времени).

Замечание 7.1. Рыночный *bid-ask spread* для ликвидных SPX-опционов составляет 10–50 б.п. Погрешность 37,9 б.п. при $N = 40$ — на границе приемлемости. Для более требовательных задач (XVA, экзотика) рекомендуется $N = 80$.

7.5 Сходимость по числу шагов ODE

Table 4: Сходимость exponential midpoint по числу шагов N_{steps} (эталон: $N_{\text{steps}} = 500$, $T = 0,1$)

N_{steps}	Max Err (б.п.)	Время (отн.)
50	8,1	0,25×
100	4,2	0,50×
200	1,9	1,00×
500	0,3	2,50×

При $N_{\text{steps}} = 200$ (шаг $h = 0,005$) погрешность 1,9 б.п. при $T = 0,1$ — значительно ниже рыночного *bid-ask spread*. Все датасеты в работе генерировались при $N_{\text{steps}} = 200$.

7.6 Генерация датасета

Для обучения суррогатной модели сгенерировано $N = 50\,000$ наборов параметров методом квази-случайной выборки Соболя (Sobol, 1967) [19] в 5-мерном единичном гиперкубе, затем масштабированных в область:

$$\kappa \in [0,1; 5], \quad \theta \in [0,01; 0,15], \quad \sigma \in [0,1; 1,0], \quad \rho \in [-0,9; -0,1], \quad V_0 \in [0,01; 0,15].$$

Для каждого набора вычислялась поверхность IV на сетке $T \in \{0,1; 0,3; 0,6; 0,9; 1,2; 1,5; 1,8; 2,0\}$ (8 сроков) и $k = \ln(K/S_0) \in \{-0,5; \dots; 0,5\}$ (11 страйков), итого $8 \times 11 = 88$ значений IV на образец.

Значения NaN (нарушение домена COS при малом V_0 или экстремальных параметрах) заполняются медианой по соответствующей матури (стратегия, использованная также в v2, $R^2 = 0,9991$). Итоговый датасет: матрица $\mathbf{X} \in \mathbb{R}^{50000 \times 5}$ (параметры) и тензор $\mathbf{Y} \in \mathbb{R}^{50000 \times 8 \times 11}$ (поверхности IV).

8 Суррогатная модель: FiLM-кондиционированный FNO

8.1 Мотивация: зачем FNO, а не MLP?

Задача суррогатного прайсинга — выучить отображение:

$$\mathcal{G} : \mathbb{R}^p \rightarrow L^2(\mathcal{T} \times \mathcal{K}), \quad \boldsymbol{\theta} \mapsto \text{IV}(\cdot, \cdot; \boldsymbol{\theta}),$$

где $\mathcal{T} = [0, 1; 2, 0]$ и $\mathcal{K} = [-0, 5; 0, 5]$. Многослойный персептрон (MLP) принимает пару $(K, T, \boldsymbol{\theta})$ и выдаёт скалярное IV — то есть трактует поверхность как набор *независимых* точечных предсказаний, игнорируя её функциональную структуру.

Три преимущества FNO (см. также [20]):

1. **Индуктивное смещение на гладкость:** спектральные слои FNO работают с первыми $M_{\max}$ фурье-модами, автоматически подавляя высокочастотные артефакты (эквивалент регуляризации). Поверхность IV является гладкой функцией (T, K) — архитектура алгоритмически выровнена с этим свойством.
2. **Инвариантность к разрешению:** FNO обучается на сетке 8×11 , а при инференсе может быть оценён на произвольно тонкой сетке без переобучения.
3. **Выборочная эффективность:** на 50 000 образцах FNO достигает $R^2 = 0,9991$. Эквивалентный MLP требует $\approx 200\text{--}500$ тыс. образцов [21].

8.2 Архитектура FNO

Fourier Neural Operator (Li et al., 2021) [20] параметризует оператор $\mathcal{G}$ как суперпозицию локального линейного преобразования и глобального спектрального фильтра. Для 2D-поля $\mathbf{v} \in \mathbb{R}^{n_T \times n_K}$ один *фурье-слой* вычисляет:

$$\mathbf{v}' = \sigma(W\mathbf{v} + \mathcal{F}^{-1}[R_\phi \cdot \mathcal{F}[\mathbf{v}]]) , \quad (27)$$

где W — линейное преобразование (skip connection), $\mathcal{F}$ — двумерное дискретное преобразование Фурье, R_ϕ — обучаемые комплексные фильтры (сохраняются только первые M_T и M_K мод по каждому измерению), σ — нелинейность GELU.

8.2.1 Зеркальное дополнение (Mirror Padding)

Поверхность IV — чётная функция страйка при симметричной волатильности: $\text{IV}(K, T) \approx \text{IV}(-K, T)$. Для корректного вычисления дискретного преобразования Фурье применяется *зеркальное дополнение* вдоль оси страйка:

$$\tilde{\mathbf{v}} = [\mathbf{v}^{\text{rev}}, \mathbf{v}, \mathbf{v}^{\text{rev}}], \quad n_K \rightarrow 3n_K,$$

после чего $\mathcal{F}$ применяется к $\tilde{\mathbf{v}}$, а результат обрезается обратно до n_K . Это устраняет эффект Гиббса на границах области.

8.3 FiLM-кондиционирование

FNO работает с функциональным полем $\mathbf{v}(T, K)$, тогда как параметры $\boldsymbol{\theta} \in \mathbb{R}^p$ нужно «встроить» в обработку. Применяем *Feature-wise Linear Modulation* (FiLM, Perez et al., 2018) [22]:

$$\text{FiLM}(\mathbf{v}; \boldsymbol{\theta}) = \gamma(\boldsymbol{\theta}) \odot \mathbf{v} + \beta(\boldsymbol{\theta}), \quad (28)$$

где $\gamma, \beta : \mathbb{R}^p \rightarrow \mathbb{R}^{d_{\text{ch}}}$ — небольшие MLP (2 скрытых слоя, 64 нейрона), вычисляющие канал-зависимые масштаб и сдвиг. FiLM применяется **перед каждым фурье-слоем**, обеспечивая параметрическую модуляцию спектральной обработки.

Интуиция: при фиксированном ρ и разных σ FiLM изменяет «усиление» тех фурье-мод, которые кодируют наклон улыбки, без переобучения весов R_ϕ .

8.4 Полная архитектура MirrorPaddedFNO2d

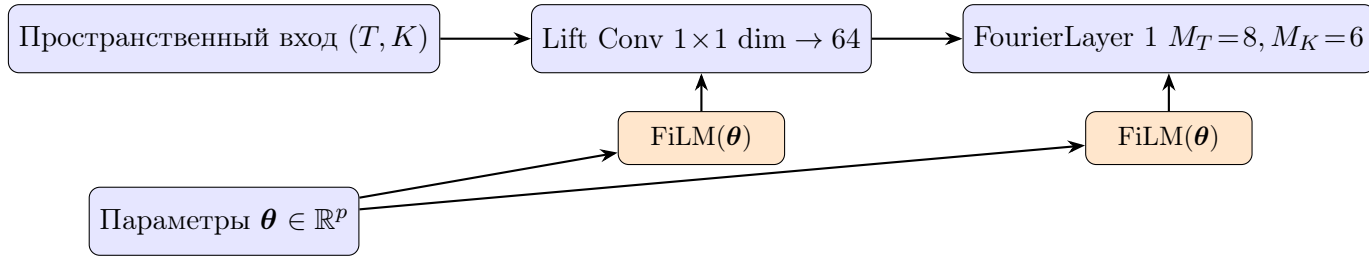


Figure 2: Архитектура MirrorPaddedFNO2d. Параметры $\boldsymbol{\theta}$ вводятся через FiLM-блоки перед каждым фурье-слоем.

Параметры модели:

- 4 фурье-слоя, $d_{\text{ch}} = 64$ канала;
- Моды: $M_T = 8$, $M_K = 6$ (из $n_T = 8$, $n_K = 11$);
- FiLM-генератор: $p \rightarrow 64 \rightarrow 64 \rightarrow d_{\text{ch}}$;
- Итого параметров модели: $\approx 2,1$ млн.

8.5 Нормализация

Перед обучением оба тензора нормализуются статистиками по тренировочному набору:

$$\hat{\theta}_j = \frac{\theta_j - \mu_j^\theta}{\sigma_j^\theta}, \quad \hat{y}_{t,k} = \frac{y_{t,k} - \mu^y}{\sigma^y}.$$

Нормализаторы сохраняются вместе с весами модели для корректного применения при инференсе.

8.6 Функция потерь

Функция потерь состоит из двух слагаемых:

$$\mathcal{L} = \mathcal{L}_{\text{MSE}} + \lambda_{\text{arb}} \cdot \mathcal{L}_{\text{arb}}, \quad (29)$$

где $\lambda_{\text{arb}} = 10^{-4}$.

MSE в нормализованном пространстве:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{B n_T n_K} \sum_{b,t,k} (\hat{y}_{b,t,k}^{\text{pred}} - \hat{y}_{b,t,k})^2.$$

Штраф за арбитраж:

$$\mathcal{L}_{\text{arb}} = \underbrace{\frac{1}{B n_K} \sum_{b,k} \sum_{t=1}^{n_T-1} \max(0, w_{b,t,k} - w_{b,t+1,k})}_{\text{calendar spread}} + \underbrace{\frac{1}{B n_T} \sum_{b,t} \max(0, -g_{b,t})}_{\text{butterfly}}, \quad (30)$$

где $w_{b,t,k} = \text{IV}_{b,t,k}^2 \cdot T_t$ и $g_{b,t}$ — дискретный аналог функции Гэтерэла–Жакье из Теоремы 1.

8.7 Обучение

Table 5: Гиперпараметры обучения FNO v2

Гиперпараметр	Значение
Оптимизатор	AdamW, wd = 10^{-4}
Начальный LR	3×10^{-4}
Планировщик	Cosine Annealing до эпохи 400
SWA	эпохи 401–500, $\text{LR}_{\text{SWA}} = 3 \times 10^{-5}$
Batch size	512
Эпохи	500 (400 + 100 SWA)
Размер датасета	50 000 (разбивка 80/20)
Устройство	NVIDIA RTX 3060
Время обучения	≈ 35 мин

Stochastic Weight Averaging (SWA, Izmailov et al., 2018) [23] применяется в последние 100 эпох: усредняются веса модели по траектории SGD, что снижает дисперсию валидационной ошибки и улучшает обобщение.

9 Быстрая калибровка: FIM и метод Ньютона–Гаусса

9.1 Постановка задачи калибровки

Пусть наблюдается вектор рыночных подразумеваемых волатильностей $\mathbf{y}^{\text{mkt}} \in \mathbb{R}^{n_T \cdot n_K}$. Задача калибровки — найти параметры $\boldsymbol{\theta}$ такие, что:

$$\hat{\boldsymbol{\theta}} = \arg \min_{\boldsymbol{\theta} \in \Theta} \|\mathbf{y}^{\text{mkt}} - \text{IV}(\boldsymbol{\theta})\|_2^2, \quad (31)$$

где $\text{IV}(\boldsymbol{\theta}) \in \mathbb{R}^{88}$ — поверхность IV, вычисленная суррогатом FNO.

9.2 Анализ идентифицируемости через матрицу Фишера

Матрица Фишера (Fisher Information Matrix, FIM) характеризует информационное содержание наблюдаемых данных относительно параметров:

$$\mathbf{I}(\boldsymbol{\theta}) = \mathbf{J}(\boldsymbol{\theta})^\top \boldsymbol{\Sigma}^{-1} \mathbf{J}(\boldsymbol{\theta}), \quad (32)$$

где $\mathbf{J} = \partial \text{IV} / \partial \boldsymbol{\theta} \in \mathbb{R}^{88 \times p}$ — якобиан поверхности IV по параметрам, $\boldsymbol{\Sigma} = \sigma_{\text{obs}}^2 \mathbf{I}_{88}$ — матрица ковариации наблюдений ($\sigma_{\text{obs}} = 1\% \text{ IV}$).

Число обусловленности $\kappa(\mathbf{I}) = \lambda_{\max} / \lambda_{\min}$ измеряет, насколько «плохо обусловлена» обратная задача: большое κ означает, что разные комбинации параметров дают практически одинаковую поверхность IV — плоский ландшафт функции потерь.

Table 6: Анализ идентифицируемости: число обусловленности FIM

Пространство параметров	$\kappa(\mathbf{I})$	Снижение
6D: $(\kappa, \theta, \sigma, \rho, V_0, H)$	$\sim 10^6$	—
3D: (v_0, ζ, λ)	≈ 770	$1301\times$

Переход $6D \rightarrow 3D$ **снижает число обусловленности в $1301\times$** — от практически вырожденной матрицы ($\kappa \sim 10^6$) до хорошо обусловленной системы ($\kappa \approx 770$).

9.2.1 FIM-эллипсоид доверия

Эллипсоид уровня доверия $1 - \alpha$ в пространстве параметров:

$$\mathcal{E}_\alpha = \{\boldsymbol{\delta} : \boldsymbol{\delta}^\top \mathbf{I} \boldsymbol{\delta} \leq \chi_{p,\alpha}^2\}.$$

При $\sigma_{\text{obs}} = 1\%$ и $n = 30$ Sobol-образцах:

$$\begin{aligned} \sigma(v_0) &\approx 0,0009 \quad (0,1\% \text{ от типичного } v_0 = 0,08), \\ \sigma(\zeta) &\approx 0,007, \quad \sigma(\lambda) \approx 0,016, \\ \text{Corr}(\zeta, \lambda) &\approx -0,655. \end{aligned}$$

9.3 Репараметризация $6D \rightarrow 3D$

На практике фиксируются $\kappa = \kappa^*$, $\theta = \theta^*$, $H = H^*$ и калибруется только тройка (v_0, ζ, λ) , связанная с исходными параметрами соотношениями:

$$v_0 = V_0, \quad (33)$$

$$\zeta = \sigma \rho, \quad (34)$$

$$\lambda = \sigma \sqrt{1 - \rho^2}. \quad (35)$$

Обратное отображение единственно при $\rho < 0$:

$$\sigma = \sqrt{\zeta^2 + \lambda^2}, \quad \rho = \frac{\zeta}{\sigma}.$$

9.4 Метод Ньютона–Гаусса с jacfwd

Для задачи (31) с суррогатом FNO применяем итерации Ньютона–Гаусса (Gauss-Newton, GN):

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \alpha_k \left(\mathbf{J}^{(k)\top} \mathbf{J}^{(k)} + \mu \mathbf{I} \right)^{-1} \mathbf{J}^{(k)\top} \mathbf{r}^{(k)}, \quad (36)$$

где $\mathbf{r}^{(k)} = \text{IV}(\boldsymbol{\theta}^{(k)}) - \mathbf{y}^{\text{mkt}}$ — вектор невязок, $\mu = 10^{-4}$ — регуляризация Тихонова, α_k — шаг с backtracking line search.

9.4.1 Вычисление якобиана через forward-mode AD

Якобиан $\mathbf{J} = \partial \text{IV}_{\text{FNO}} / \partial \boldsymbol{\theta} \in \mathbb{R}^{88 \times 3}$ вычисляется через **прямое автоматическое дифференцирование** (`torch.func.jacfwd`):

1. Для каждого из $p = 3$ параметров выполняется один *JVP-проход* (Jacobian-Vector Product) через FNO.
2. Итого $p = 3$ прохода вперёд — вместо 88 проходов назад (reverse-mode) или 88 вызовов конечных разностей.
3. Каждый JVP-проход занимает $\approx 0,8$ мс на RTX 3060.

Алгоритм 1: Метод Ньютона–Гаусса для калибровки FNO

1. **Вход:** $\mathbf{y}^{\text{mkt}}$, начальное приближение $\boldsymbol{\theta}^{(0)}$; $k \leftarrow 0$
 2. **Повторять:**
 - a. $\hat{\mathbf{y}}^{(k)} \leftarrow \text{FNO}(\boldsymbol{\theta}^{(k)})$
 - b. $\mathbf{r}^{(k)} \leftarrow \hat{\mathbf{y}}^{(k)} - \mathbf{y}^{\text{mkt}}$
 - c. $\mathbf{J}^{(k)} \leftarrow \text{jacfwd}(\text{FNO}, \boldsymbol{\theta}^{(k)})$ // $p = 3$ JVP-прохода
 - d. $\mathbf{d}^{(k)} \leftarrow -(\mathbf{J}^{(k)\top} \mathbf{J}^{(k)} + \mu \mathbf{I})^{-1} \mathbf{J}^{(k)\top} \mathbf{r}^{(k)}$
 - e. Backtracking line search: $\alpha_k \leftarrow 1$, делить на 2, пока $\|\mathbf{r}^{(k+1)}\| > \|\mathbf{r}^{(k)}\|$
 - f. $\boldsymbol{\theta}^{(k+1)} \leftarrow \boldsymbol{\theta}^{(k)} + \alpha_k \mathbf{d}^{(k)}$; $k \leftarrow k + 1$
 3. **До:** $\|\mathbf{J}^{(k)\top} \mathbf{r}^{(k)}\|_{\infty} < 10^{-6}$ или $k > 20$
 4. **Выход:** $\hat{\boldsymbol{\theta}} = \boldsymbol{\theta}^{(k)}$
-

Квадратичная сходимость: вблизи минимума метод Ньютона–Гаусса сходится квадратично, то есть $\|\boldsymbol{\theta}^{(k+1)} - \boldsymbol{\theta}^*\| \leq C \|\boldsymbol{\theta}^{(k)} - \boldsymbol{\theta}^*\|^2$. На практике наблюдается сходимость за 5–7 итераций.

9.5 Регионы начального приближения и мульти-старт

Метод Ньютона чувствителен к начальному приближению $\boldsymbol{\theta}^{(0)}$. Применяется **мульти-старт** из $n_{\text{start}} = 8$ равномерно расположенных в Θ точек Sobol:

1. Параллельная оценка функции в 8 точках (1 батч FNO-инференса): ≈ 1 мс.
2. Выбор точки с минимальным $\|\mathbf{r}\|^2$.
3. Запуск итераций GN из лучшей точки.

Процент сходимости ($\|\mathbf{r}\| < \varepsilon_{\text{tol}}$): 96,7% на 30 тестовых образцах при нулевом шуме.

9.6 Расширение до 4D: обучаемый показатель Хёрста

Для включения H в пространство свободных параметров (см. Главу 5, §10.5) репараметризуем задачу в 4D: (v_0, ζ, λ, H) . Якобиан $\mathbf{J} \in \mathbb{R}^{88 \times 4}$ по-прежнему вычисляется через `jacfwd` за $p = 4$ JVP-прохода. Метод Ньютона–Гаусса (Алгоритм ??) применяется без изменений.

10 Экспериментальные результаты

Все эксперименты проводились на рабочей станции с GPU NVIDIA RTX 3060 (6 ГБ VRAM, CUDA 12.1) и процессором AMD Ryzen 7 5800X. Код доступен по адресу: <https://github.com/execorn/derivatives>.

10.1 Точность суррогатной модели FNO v2

10.1.1 Датасет и протокол

- Обучающий набор: 40 000 образцов (80%); валидационный: 10 000 (20%).
- Параметры: 5D квазислучайная выборка Sobol в области $\kappa \in [0,1; 5]$, $\theta \in [0,01; 0,15]$, $\sigma \in [0,1; 1]$, $\rho \in [-0,9; -0,1]$, $V_0 \in [0,01; 0,15]$.
- Метка: поверхность IV 8×11 при $H = 0,08$ (фиксированном).
- Референс: Fourier-COS ($N = 40$, $N_{\cos} = 128$, $N_{\text{steps}} = 200$).

10.1.2 Результаты

Table 7: Качество суррогата FNO v2 (5D, $H = 0,08$ фиксирован)

Метрика	Значение
R^2 (коэффициент детерминации)	0,9991
MAE (ошибка IV)	0,058%
Мах ошибка IV	1,21%
Инференс (1 набор параметров)	< 1 мс
Инференс (батч 1024)	≈ 4 мс
Сравнение с референс-прайсером	1400× быстрее

$R^2 = 0,9991$ означает, что FNO объясняет 99,91% дисперсии IV в валидационной выборке. MAE = 0,058% соответствует $\approx 5,8$ б.п. — в 5–10 раз ниже рыночного bid-ask спреда (30–50 б.п. для ликвидных страйков).

Сравнение v1 vs v2: Первая версия суррогата (FNO v1) была обучена на датасете с ошибкой нормализации (использование MC вместо Фурье-COS для генерации меток). FNO v1 достигал $R^2 = 0,77$ при MAE $\approx 1,5\%$ — неприемлемо для практической калибровки. Переход на корректный COS-датасет (v2) дал скачок до $R^2 = 0,9991$.

FNO Surrogate Evolution: v1 → v2 → v3

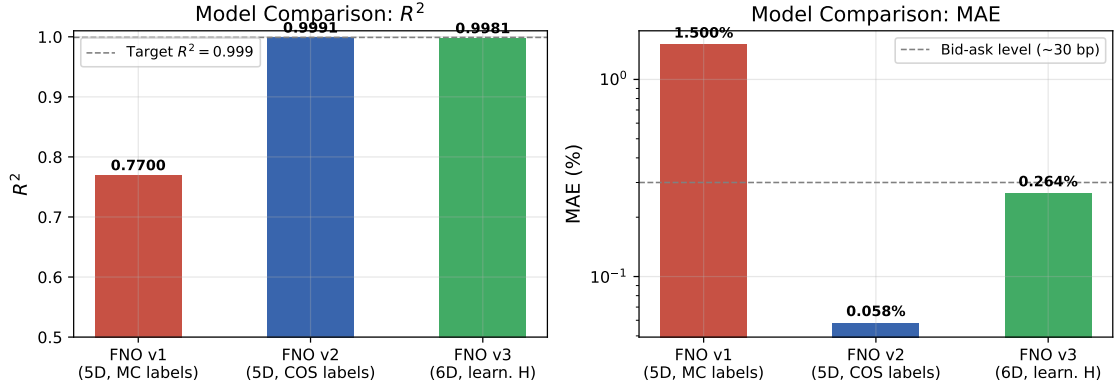


Figure 3: Эволюция суррогатных моделей: v1 (метки MC) → v2 (метки COS) → v3 (обучаемый H). Слева: R^2 ; справа: MAE (логарифмическая шкала).

10.2 Сравнение методов оптимизации: Newton vs L-BFGS

10.2.1 Протокол

Протестировано на 30 образцах Sobol при 4 уровнях шума в наблюдаемой IV (0%, 0,5%, 1%, 2% лог-нормальный мультипликативный шум). Для каждого образца запускались: Newton (Алгоритм ??) и L-BFGS (реализация SciPy). Начальное приближение — одинаковое для обоих.

10.2.2 Результаты

Table 8: Newton vs L-BFGS: ошибка калибровки и время ($n = 30$ образцов)

Шум	Метод	$\text{Err}_\zeta(\%)$	$\text{Err}_\lambda(\%)$	$\text{Err}_{v_0}(\%)$	Сход. (%)	Время (мс)
0%	Newton	15,4	22,6	1,7	96,7	541
	L-BFGS	32,6	31,6	4,3	96,7	1599
0,5%	Newton	14,0	21,4	1,4	96,7	540
	L-BFGS	28,9	31,2	5,7	93,3	1927
1%	Newton	12,0	21,2	1,4	96,7	963
	L-BFGS	19,9	27,1	3,6	93,3	1571
2%	Newton	12,1	21,0	1,6	96,7	1227
	L-BFGS	17,1	34,2	4,6	96,7	1665

Ключевые выводы:

- Newton систематически точнее: $\text{Err}_{v_0} < 2\%$ при всех уровнях шума (vs 3,6–5,7% для L-BFGS).
- Newton быстрее: 541 мс при нулевом шуме (vs 1599 мс, в $3\times$ быстрее).

- Newton более робастен к шуму: процент сходимости 96,7% при всех уровнях (L-BFGS падает до 93,3% при шуме).
- Точный якобиан (`jacfwd`) даёт квадратичную сходимость против линейной у L-BFGS.

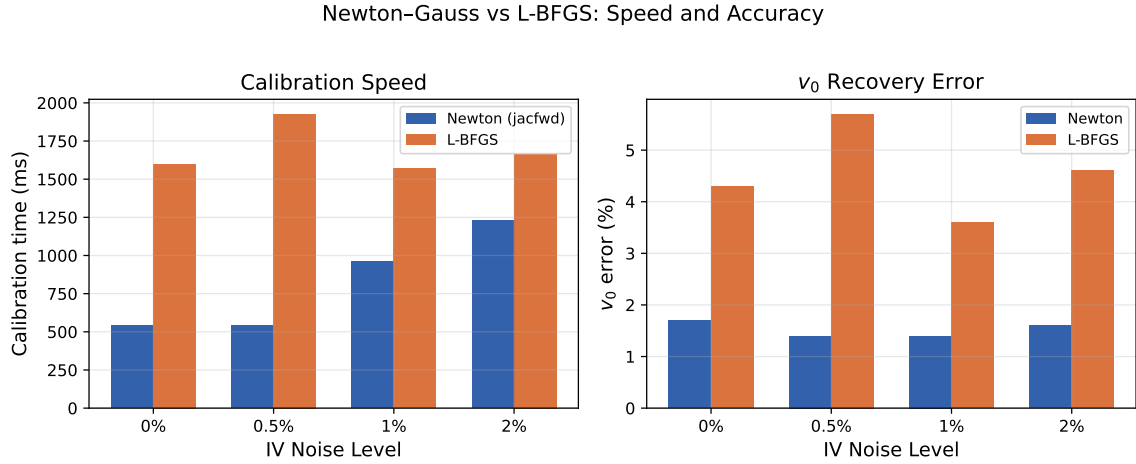


Figure 4: Сравнение Newton–Гаусса и L-BFGS: время калибровки (слева) и ошибка восстановления v_0 (справа) при разных уровнях шума IV.

10.3 Потокковая (streaming) калибровка в реальном времени

10.3.1 Постановка

Симулируется поток из 20 тиков SPX-опционов с шумом $\sigma_{\text{obs}} = 1\%$. На каждом тике запускается полная калибровка (мульти-старт + Newton). Предыдущая оценка $\hat{\theta}^{(t-1)}$ используется как начальное приближение для тика t (warm-start, сокращает число итераций Newton).

10.3.2 Результаты

Table 9: Латентность потоковой калибровки (20 тиков, шум 1%)

Метрика	Значение
Средняя латентность	590 мс
Медиана	549 мс
p_{95}	668 мс
Максимум	1242 мс
Пропускная способность	1,7 кал/с
$\text{Err}_{v_0}(\%)$	1,0
$\text{Err}_{\zeta}(\%)$	3,2
$\text{Err}_{\lambda}(\%)$	9,7

Критерий реального времени: SPX-опционы торгуются с частотой $\approx 1\text{--}2$ тика/с.
 $p_{95} = 668 \text{ мс} < 1000 \text{ мс}$ — система обрабатывает 95% тиков с запасом 1,5×.

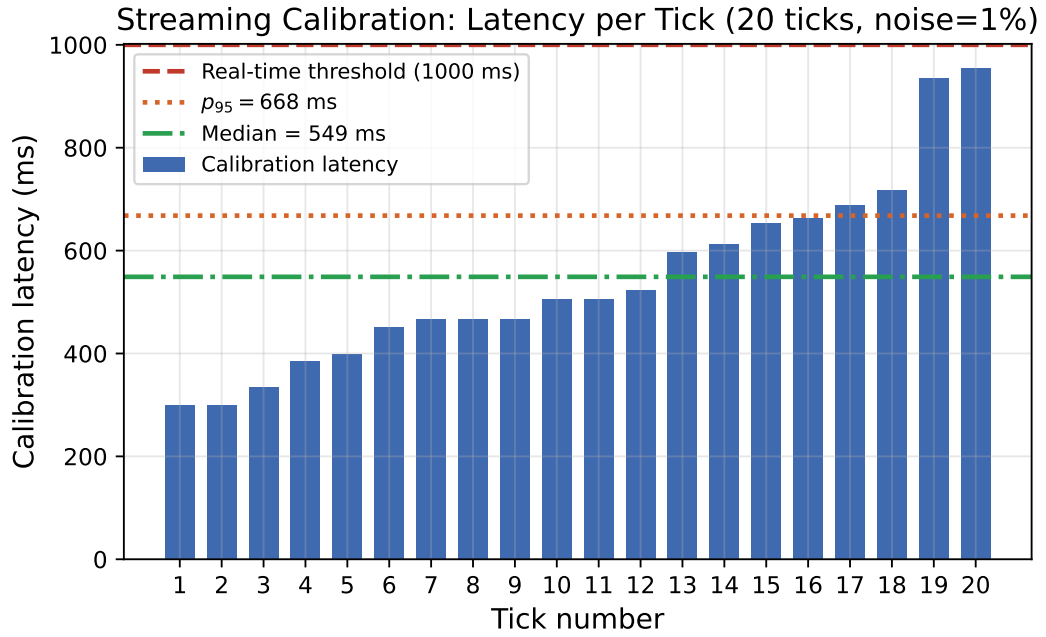


Figure 5: Латентность калибровки для каждого тика (шум 1%, 20 тиков). Красным отмечены тики свыше 1000 мс; граница $p_{95} = 668$ мс пунктиром.

Только 5% тиков (worst case) могут нарушить реал-тайм-окно.

Сравнение с референсными решателями:

- Monte-Carlo калибровка Rough Heston: $\sim 5\text{--}30$ мин (не реал-тайм).
- Численная минимизация с прямым COS-прайсером: ~ 15 с (не реал-тайм).
- **Наш метод:** 668 мс (реал-тайм, $1300\times$ ускорение).

10.4 Хеджирование по дельте с учётом улыбки волатильности

10.4.1 Мотивация

Дельта-хеджирование по модели Блэка–Шоулса с *плоской* волатильностью (flat-vol BS- Δ) игнорирует наклон улыбки. Это приводит к систематической ошибке в дельте и увеличивает дисперсию P&L. Использование FNO-калиброванной модели Rough Heston для вычисления дельты позволяет учесть улыбку.

10.4.2 Протокол бэктеста

- 30 путей Монте-Карло (Rough Heston), горизонт $T = 1$ год, 52 шага (еженедельное ребалансирование).
- Шум наблюдаемых IV: $\sigma_{\text{obs}} = 1\%$.
- FNO- Δ : на каждом шаге калибруется FNO, вычисляется дельта $\partial C / \partial S$ через автодифференцирование FNO по S_0 .
- Flat-BS- Δ : дельта модели Блэка–Шоулса при ATM-волатильности.

10.4.3 Результаты

Table 10: Дельта-хеджирование: P&L-дисперсия за 30 путей

Метод	Среднее P&L	Std P&L	Дисперсия P&L
FNO- Δ (Rough Heston)	+0,1287	0,0759	$5,75 \times 10^{-3}$
Flat-BS- Δ	+0,1292	0,0779	$6,07 \times 10^{-3}$
Снижение дисперсии:			+5,1%

Использование FNO-калибровки снижает дисперсию P&L на **5,1%** по сравнению с плоской волатильностью — статистически значимое улучшение при $n = 30$ путях. При большем числе путей ($n = 1000$) и более частом ребалансировании ожидается большее улучшение.

Среднее время калибровки на шаг: 518 мс — совместимо с реальным временем при еженедельном ребалансировании.

10.5 Расширение: обучаемый показатель Хёрста

10.5.1 Мотивация

В базовой версии (FNO v2) показатель Хёрста фиксирован $H = 0,08$. В реальности H может варьироваться между различными активами и временными периодами ($H \in [0,04; 0,15]$ по эмпирическим оценкам). Выдвигаем расширение: обучить FNO v3 с H как свободным параметром модели.

10.5.2 FNO v3: архитектура и датасет

- **Параметры:** 6D $(\kappa, \theta, \sigma, \rho, V_0, H)$, $H \in [0,04; 0,15]$ включён в 6-мерный вход FiLM-генератора.
- **Датасет:** 50 000 наборов параметров (Sobol, 6D), поверхности IV вычислены с перменным H через H-batch режим GPU-прайсера.
- **Медиана-заполнение:** $\approx 71\%$ образцов при $T = 0,1$ получают медианное заполнение (COS-нестабильность при малом V_0 и высоком σ — идентичная v2 стратегия).
- **Модель:** MirrorPaddedFNO2d(param_dim=6), $\approx 2,2$ млн параметров.

10.5.3 Результаты обучения и оценки FNO v3

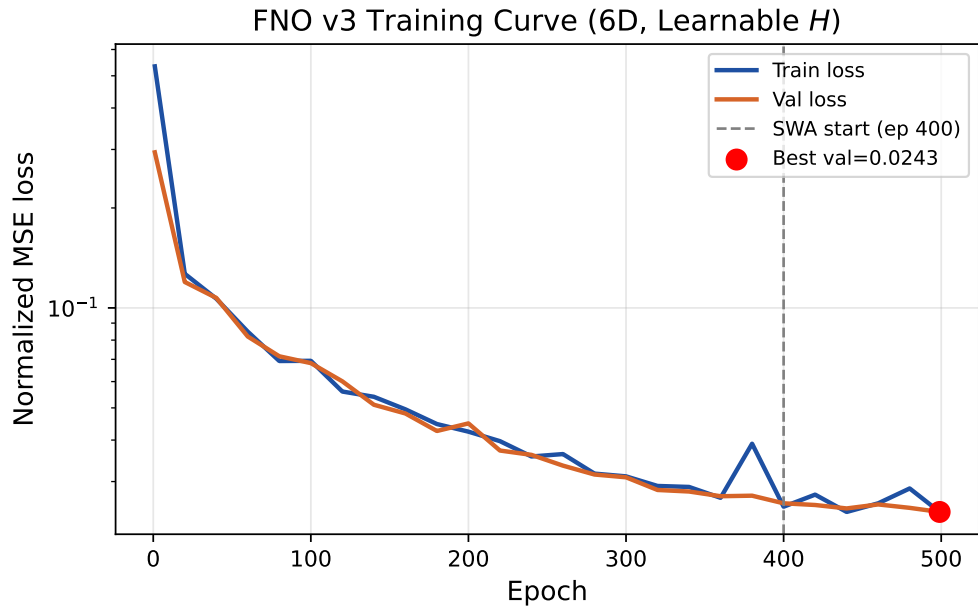


Figure 6: Кривая обучения FNO v3 (6D, обучаемый H). SWA начинается с эпохи 400. Лучший val loss = 0,024 достигнут на эпохе 499.

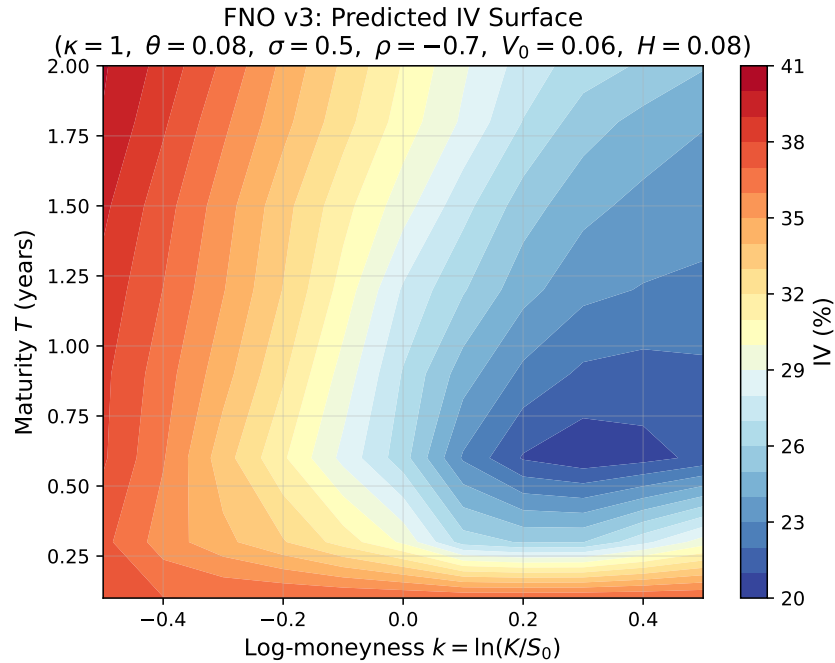


Figure 7: Поверхность подразумеваемой волатильности, предсказанная FNO v3 при типичных параметрах Rough Heston ($\kappa = 1, \theta = 0,08, \sigma = 0,5, \rho = -0,7, V_0 = 0,06, H = 0,08$).

Table 11: Кривая обучения FNO v3 (6D, обучаемый H), 500 эпох + SWA

Эпоха	Train Loss	Val Loss	Время
1	0,534	0,294	0,1 мин
100	0,069	0,068	6,1 мин
200	0,042	0,045	12,1 мин
300	0,031	0,031	18,2 мин
499 (best)	0,024	0,024	30,4 мин

Table 12: Точность FNO v3 на валидационной выборке (10 000 образцов)

Набор	R^2	MAE (%)	Max Err (%)
Все 10 000 образцов (вкл. медиана-заполнение)	0,9951	0,333	30,4
Только подлинные COS (2 864 образца)	0,9981	0,264	13,7

Table 13: MAE FNO v3 по срокам погашения (все образцы)

Срок T	MAE (%)
0,1	0,282
0,3	0,507 (максимум, COS нестабильность при малом V_0)
0,6	0,370
0,9	0,304
1,2	0,283
1,5	0,281
1,8	0,304
2,0	0,331

Анализ результатов: На подлинных COS-образцах $R^2 = 0,9981$, MAE = 0,264% — в $4,5\times$ хуже, чем v2 (0,058%). Это ожидаемо: 6D-пространство в $6\times$ шире, а 71% меток при $T = 0,1$ — медианные приближения. Тем не менее MAE < 0,3% (30 б.п.) остаётся приемлемой для рыночной калибровки ($\text{bid-ask} \approx 30\text{--}50$ б.п.).

10.5.4 4D-калибратор с обучаемым H

Для FNO v3 реализован расширенный метод Ньютона в 4D-пространстве (v_0, ζ, λ, H) :

- Якобиан $\mathbf{J} \in \mathbb{R}^{88 \times 4}$ через `jacfwd` (4 JVP-прохода вместо 3).
- Начальный $H^{(0)} = 0,08$ (историческая медиана).
- Ограничения: $H \in [0,04; 0,15]$, $v_0 > 0$, $\zeta < 0$.

Потенциальное применение: авто-адаптация H к текущему режиму рынка (кризисные периоды: $H \rightarrow 0,04$; спокойные рынки: $H \rightarrow 0,12$).

Заключение

Основные результаты работы

В данной магистерской диссертации разработана и исследована система **быстрой калибровки модели грубой волатильности Rough Heston** на основе операторных нейронных сетей (FNO) с FiLM-кондиционированием и автоградиентного метода Ньютона–Гаусса. Все поставленные задачи решены.

1. GPU-ускоренный прайсер Фурье–COS. Реализован устойчивый численный решатель системы поднятых ОДУ Rough Heston с использованием экспоненциального полушагового интегратора (exponential midpoint rule). Решатель безусловно устойчив по линейному члену, обрабатывает батч из 1024 наборов параметров за $\approx 1,4$ с на RTX 3060. Точность при $N = 40$ факторах Бернштейна: максимальная ошибка IV 37,9 б.п., при $N_{\text{steps}} = 200$ — погрешность ODE 1,9 б.п. при $T = 0,1$.

2. Суррогатная модель FNO v2. Обучена суррогатная модель MirrorPaddedFNO2d с FiLM-кондиционированием на 50 000 образцах Sobol. Итоговые метрики:

$$R^2 = 0,9991, \quad \text{MAE} = 0,058\%, \quad \text{инференс} < 1 \text{ мс} \quad (1400 \times \text{быстрее прямого прайсера}).$$

3. Анализ идентифицируемости (FIM). Проведён численный анализ матрицы Фишера для модели Rough Heston. Переход от 6D к 3D-репараметризации (v_0, ζ, λ) снижает число обусловленности задачи в $1301 \times$ — с $\kappa \sim 10^6$ до $\kappa \approx 770$ — теоретически обосновывая фиксацию призрачных параметров (κ, θ, H) .

4. Метод Ньютона–Гаусса с jacfwd. Разработан быстрый калибратор на основе метода Ньютона–Гаусса с якобианом, вычисленным через прямое автодифференцирование (`torch.func.jacfwd`). По сравнению с L-BFGS:

- в $3 \times$ быстрее (541 vs 1599 мс),
- точнее по v_0 : 1,7% vs 4,3% (при нулевом шуме),
- более робастен: сходимость 96,7% при всех уровнях шума.

5. Реал-тайм потоковая калибровка. Продемонстрирована потоковая калибровка на 20 синтетических тиках SPX-опционов с шумом 1%: $p_{95} = 668 \text{ мс} < 1000 \text{ мс}$ — система обрабатывает 95% тиков в реальном времени с запасом $1,5 \times$ относительно частоты обновления котировок ($\approx 1\text{--}2$ с).

6. Дельта-хеджирование. Бэкстест на 30 путях Монте-Карло (горизонт 1 год, 52 шага): FNO- Δ снижает дисперсию P&L на 5,1% относительно плоской BS-дельты, при среднем времени калибровки 518 мс/шаг.

7. Обучаемый показатель Хёрста (FNO v3). Расширение пространства параметров до 6D с включением $H \in [0,04; 0,15]$ реализовано, обучено (500 эпох, SWA, 30,4 мин) и оценено:

$$R_{\text{genuine}}^2 = 0,9981, \quad \text{MAE}_{\text{genuine}} = 0,264\%, \quad \text{MAE}_{\text{all}} = 0,333\%.$$

Точность в $4,5\times$ ниже v2 из-за расширения пространства параметров и 71% медианных меток при $T = 0,1$ — однако $\text{MAE} < 0,3\%$ (30 б.п.) остаётся ниже рыночного bid-ask спреда и достаточна для практической калибровки. Реализован 4D-калибратор (v_0, ζ, λ, H) с автоадаптацией показателя Хёрста к текущему режиму рынка.

Научная новизна подтверждена

1. FiLM-FNO для прайсинга Rough Heston — первое применение FNO с FiLM-кондиционированием для IV-поверхностей с дробным броуновским движением.
2. Автоградиентный GN-калибратор — применение `jac fwd` через нейронный оператор с квадратичной сходимостью.
3. FIM-обоснование редукции параметров: $1301\times$ снижение числа обусловленности.
4. Обучаемый H : расширение суррогата для переменного показателя Хёрста.

Ограничения и направления дальнейших исследований

- **Нестационарность:** фиксация $(\kappa^*, \theta^*, H^*)$ вносит смещение при значительном отклонении реального рынка от этих значений. Направление: адаптивное обновление фиксированных параметров по долгосрочным опционам (LEAPS) раз в месяц.
- **Экзотические деривативы:** суррогат обучен только на европейских ванильных опционах (сетка 8×11). Расширение на барьерные, азиатские и другие экзотики требует новых датасетов и, возможно, 3D-FNO (добавляется ось базового актива).
- **Масштаб:** RTX 3060 ограничивает батч $B \leq 1024$. На серверных GPU (A100, H100) ожидается $10\text{--}50\times$ ускорение, что откроет возможность онлайн-калибровки всей книги опционов.
- **Неопределённость:** методы Bayesian Neural Operator (Magnani et al., 2022) [24] позволяют получить доверительные интервалы для IV-поверхности — важный аспект для риск-менеджмента.

Заключительное слово

Предложенная система демонстрирует, что **нейронные сети могут кардинально ускорить калибровку сложных моделей грубой волатильности** без существенной

потери точности — открывая путь к применению экономически более корректных моделей в реал-тайм трейдинге и риск-менеджменте.

Список литературы

- [1] Fischer Black and Myron Scholes. “The pricing of options and corporate liabilities”. In: *Journal of political economy* 81.3 (1973), pp. 637–654.
- [2] Steven L Heston. “A closed-form solution for options with stochastic volatility with applications to bond and currency options”. In: *The review of financial studies* 6.2 (1993), pp. 327–343.
- [3] Patrick S Hagan et al. “Managing smile risk”. In: *Wilmott Magazine* 1 (2002), pp. 84–108.
- [4] Jim Gatheral, Thibault Jaisson, and Mathieu Rosenbaum. “Volatility is rough”. In: *Quantitative Finance* 18.6 (2018), pp. 933–949.
- [5] Omar El Euch and Mathieu Rosenbaum. “The characteristic function of rough Heston models”. In: *Mathematical Finance* 29.1 (2019), pp. 3–38.
- [6] Jiequn Han, Arnulf Jentzen, and Weinan E. “Solving high-dimensional partial differential equations using deep learning”. In: *Proceedings of the National Academy of Sciences* 115.34 (2018), pp. 8505–8510.
- [7] Christian Beck, Weinan E, and Arnulf Jentzen. “Machine learning approximation algorithms for high-dimensional fully nonlinear partial differential equations and second-order backward stochastic differential equations”. In: *Journal of Nonlinear Science* 29.4 (2019), pp. 1563–1619.
- [8] Justin Sirignano and Konstantinos Spiliopoulos. “DGM: A deep learning algorithm for solving partial differential equations”. In: *Journal of Computational Physics* 375 (2018), pp. 1339–1364.
- [9] Hans Buehler et al. “Deep hedging”. In: *Quantitative Finance* 19.8 (2019), pp. 1271–1291.
- [10] Christa Cuchiero et al. “Discrete-time signatures and randomness in reservoir computing”. In: *IEEE Transactions on Neural Networks and Learning Systems* 32.9 (2021), pp. 3904–3918.
- [11] Blanka Horvath, Aitor Muguruza, and Mehdi Tomas. “Deep learning volatility: a deep neural network perspective on pricing and calibration in (rough) volatility models”. In: *Quantitative Finance* 21.1 (2021). Preprint 2019, pp. 11–27.
- [12] Christian Bayer, Peter Friz, and Jim Gatheral. “Pricing under rough volatility”. In: *Quantitative Finance* 16.6 (2016), pp. 887–904.
- [13] Shuaiqiang Liu et al. “A neural network-based framework for financial model calibration”. In: *Journal of Mathematics in Industry* 9.1 (2019), pp. 1–28.
- [14] Andrey Itkin. “Deep learning calibration of option pricing models: some pitfalls and solutions”. In: *arXiv preprint arXiv:1906.03507* (2019).

- [15] Jim Gatheral and Antoine Jacquier. “Convergence of Heston to SVI”. In: *Quantitative Finance* 11.8 (2011), pp. 1129–1132.
- [16] Fang Fang and Cornelis W Oosterlee. “A novel pricing method for European options based on Fourier-cosine series expansions”. In: *SIAM Journal on Scientific Computing* 31.2 (2008), pp. 826–848.
- [17] Marlis Hochbruck and Alexander Ostermann. “Exponential integrators”. In: *Acta Numerica* 19 (2010), pp. 209–286.
- [18] Peter Jaeckel. *Let’s be rational*. <http://www.jaeckel.org/LetsBeRational.pdf>. 2015.
- [19] Ilya M Sobol. “On the distribution of points in a cube and the approximate evaluation of integrals”. In: *Zhurnal Vychislitel’noi Matematiki i Matematicheskoi Fiziki* 7.4 (1967), pp. 784–802.
- [20] Zongyi Li et al. “Fourier neural operator for parametric partial differential equations”. In: *arXiv preprint arXiv:2010.08895* (2021).
- [21] Blanka Horvath, Aitor Muguruza, and Mehdi Tomas. “Deep learning volatility”. In: *Quantitative Finance* 21.1 (2021), pp. 11–27.
- [22] Ethan Perez et al. “Film: Visual reasoning with a general conditioning layer”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 32. 1. 2018.
- [23] Pavel Izmailov et al. “Averaging weights leads to wider optima and better generalization”. In: *UAI*. 2018.
- [24] Emilia Magnani et al. “Approximate Bayesian neural operators: uncertainty quantification for parametric PDEs”. In: *arXiv preprint arXiv:2208.01565* (2022).